

UNIVERSIDAD POLITÉCNICA DE MADRID

ESCUELA TÉCNICA SUPERIOR DE INGENIEROS DE TELECOMUNICACIÓN



MÁSTER UNIVERSITARIO EN INGENIERÍA DE
TELECOMUNICACIÓN

TRABAJO FIN DE MÁSTER

DESARROLLO DE UNA PLATAFORMA DE
COMUNICACIÓN CON PERIFÉRICOS SOBRE
MPSOC DE ALTAS PRESTACIONES PARA
ORDENADORES DE A BORDO EN
SISTEMAS ESPACIALES

JORGE ALEJANDRO ESTEFANÍA HIDALGO

MÁSTER UNIVERSITARIO EN INGENIERÍA DE TELECOMUNICACIÓN

TRABAJO FIN DE MÁSTER

Título: Desarrollo de una plataforma de comunicación con periféricos sobre MPSoC de altas prestaciones para Ordenadores de a Bordo en sistemas espaciales.

Autor: D. Jorge Alejandro Estefanía Hidalgo

Tutor: D. Daniel Sánchez García

Ponente: D. Álvaro Araujo Pinto

Departamento: Departamento de Ingeniería Electrónica

MIEMBROS DEL TRIBUNAL

Presidente: D.

Vocal: D.

Secretario: D.

Suplente: D.

Los miembros del tribunal arriba nombrados acuerdan otorgar la calificación de:
.....

Madrid, a de de 20...

UNIVERSIDAD POLITÉCNICA DE MADRID

ESCUELA TÉCNICA SUPERIOR DE INGENIEROS DE TELECOMUNICACIÓN



MÁSTER UNIVERSITARIO EN INGENIERÍA DE
TELECOMUNICACIÓN

TRABAJO FIN DE MÁSTER

**Desarrollo de una plataforma de
comunicación con periféricos sobre
MPSoC de altas prestaciones para
Ordenadores de a Bordo en sistemas
espaciales**

Jorge Alejandro Estefanía Hidalgo

RESUMEN

Este Trabajo de Fin de Máster presenta el desarrollo de una plataforma de comunicación con periféricos sobre el MPSoC Zynq UltraScale+ ZCU102, en el marco del proyecto LINCE, una iniciativa del PERTE Aeroespacial liderada por Indra para el desarrollo de microsátélites de órbita baja.

El trabajo abarca cuatro áreas principales.

En primer lugar, el diseño e implementación en VHDL de un transceptor serie altamente configurable para la lógica programable del MPSoC.

En segundo lugar, el desarrollo de un driver serie en C sobre el sistema operativo en tiempo real RTEMS en el sistema de procesamiento del MPSoC, que utiliza el transceptor serie desarrollado y ofrece una API pública que abstrae completamente del hardware. En tercer lugar, el diseño y fabricación de tres placas de circuito impreso de ampliación de la ZCU102 que se conectan a ella para expandir sus capacidades y permiten ampliar la ventana de desarrollo. La primera placa ofrece soporte hardware para 7 líneas RS422 y 7 líneas RS485, las cuales se pueden conectar en buses compartidos configurables por hardware. La segunda placa ofrece soporte para 3 líneas RS422 o RS485 seleccionable, dos líneas CAN, cuatro líneas PWM y cuatro líneas analógicas. La tercera placa ofrece soporte para cinco líneas RS422 o RS485 seleccionable, tres pares de líneas de potencia para controlar motores por PWM y dos líneas SpaceWire.

En cuarto lugar, la validación de la plataforma mediante el desarrollo de herramientas de software y hardware adicionales, que permitieron verificar el correcto funcionamiento del driver serie y de las distintas funcionalidades de las placas de ampliación.

El conjunto del trabajo constituye una plataforma funcional y validada que el equipo de Sener, Indra y el laboratorio B105 pueden utilizar como base para el subsistema de comunicaciones del ordenador de a bordo del satélite LINCE en el futuro.

SUMMARY

[Pendiente de redacción — máximo 500 palabras.]

PALABRAS CLAVE

MPSoC, FPGA, VHDL, RTEMS, RS422, RS485, CAN, SpaceWire, driver serie, PCB, sistemas empotrados, tiempo real, satélite, ordenador de a bordo.

KEYWORDS

[Pendiente de redacción.]

Agradecimientos

...

Índice

Resumen y Palabras Clave	II
Agradecimientos	IV
Lista de acrónimos	VIII
1. Introducción y objetivos	1
1.1. Introducción y motivación	1
1.2. Metodología	1
1.3. Objetivos	2
2. Marco teórico	4
2.1. Proyecto LINCE	4
2.2. MPSoC Zynq UltraScale+ ZCU102	4
2.3. Vivado y Vitis (Xilinx): PS y PL	5
2.3.1. Vivado Design Suite	5
2.3.2. Vitis Unified Software Platform	5
2.4. RTEMS	6
2.5. Comunicaciones serie	6
2.5.1. RS485	6
2.5.2. RS422	7
2.5.3. SpaceWire	7
2.5.4. CAN	7
3. Desarrollo	9
3.1. Preparación del entorno de desarrollo	9
3.1.1. Instalación de Vivado y Vitis	9
3.1.2. Instalación de RTEMS	9
3.1.3. Flujo de desarrollo completo	9
3.2. Diseño del transceptor serie configurable (PL)	11
3.2.1. Transmisor	11
3.2.2. Receptor	11
3.2.3. Oscilador controlado numéricamente (NCO)	12
3.2.4. Shift-register	12
3.2.5. FIFO	12
3.2.6. Bloque Zynq UltraScale+ MPSoC	12
3.3. Herramientas para facilitar el desarrollo del transceptor (PL)	13
3.3.1. Generación de transceivers con TCL	13
3.3.2. Generación de proyecto con TCL	13
3.4. Generación del binario en Vitis (PS)	13
3.5. Diseño del driver serie en RTEMS (PS)	13
3.5.1. Motivación y contexto	13

3.5.2.	Interfaz hardware: mapa de registros	14
3.5.3.	Arquitectura software	14
3.5.4.	Descubrimiento dinámico del hardware	14
3.5.5.	Modelo de interrupciones: ISR maestra y tareas worker	15
3.5.6.	Inicialización y configuración del hardware	15
3.5.7.	API pública	15
3.5.8.	Decisiones de diseño destacadas	16
3.6.	Diseño hardware	16
3.6.1.	Diseño de la placa de comunicación serie (diseño propio)	16
3.6.2.	Diseño de la placa CDHS	18
3.6.3.	Diseño de la placa AOCS	20
3.6.4.	Fabricación	21
3.7.	Desarrollo de herramientas de testing	22
3.7.1.	Aplicación de testing del driver serie en RTEMS	22
3.7.2.	Aplicación de testing de las placas CDHS y AOCS	23
3.7.3.	Validación de hardware: arneses y equipamiento	24
4.	Resultados	25
4.1.	Montaje del sistema	25
4.2.	Validación de comunicaciones serie RS422/RS485	25
4.2.1.	CDHS — 3 transceptores	26
4.2.2.	AOCS — 5 transceptores	26
4.2.3.	Prueba con 13 transceptores simultáneos	27
4.3.	Validación del bus CAN	27
4.4.	Validación del ADC SPI (termistores CDHS)	28
4.5.	Validación PWM (calentadores CDHS y puentes en H AOCS)	29
5.	Conclusiones y líneas futuras	30
5.1.	Conclusiones	30
5.2.	Líneas futuras	30
	Bibliografía	32
	Anexo 1: Mapas de señales y tabla NCO	33
	Anexo A: Aspectos éticos, económicos, sociales y ambientales	38
	Anexo B: Presupuesto económico	40

Índice de figuras

3.1. Diagrama FSM del transmisor serie.	11
3.2. Diagrama FSM del receptor serie.	11
3.3. Circuito del oscilador controlado numéricamente (NCO).	12
3.4. Arquitectura general de la placa de comunicación serie.	17
3.5. Diagrama de bloques de la placa CDHS.	18
3.6. Render 3D de la placa CDHS.	18
3.7. Subsistema CAN de la placa CDHS.	19
3.8. Canal RS de la placa CDHS con THVD1424RGTR.	19
3.9. Diagrama de bloques de la placa AOCS.	20
3.10. Render 3D de la placa AOCS.	20
3.11. Subsistema SpaceWire de la placa AOCS.	21
3.12. Aplicación de pasta de soldadura con stencil.	22
3.13. Placa CDHS tras el proceso de soldadura.	22
3.14. Placa AOCS tras el proceso de soldadura.	22
4.1. Sistema completo montado: ZCU102 con la placa CDHS conectada al FMC.	25
4.2. Placa CDHS con componentes montados (vista superior).	25
4.3. Placa AOCS con componentes montados (vista superior).	25
4.4. Captura completa del terminal de pruebas RS en la placa CDHS.	26
4.5. Captura completa del terminal de pruebas RS en la placa AOCS.	27
4.6. Señal diferencial CAN sin resistencias de terminación.	28
4.7. Señal diferencial CAN con resistencias de terminación ($60\ \Omega + 60\ \Omega$).	28
4.8. Captura completa del terminal de lectura ADC (placa CDHS).	28
4.9. Señal PWM medida en el conector J5 de la placa CDHS.	29
4.10. Señales PWM de control de motor en la placa AOCS (par complementario para puente en H).	29

Índice de tablas

3.1. API pública del driver serie.	16
3.2. Control complementario DE/RE en el THVD1424.	19
5.1. Mapa de señales entre la placa LINCE Comunicación Serial y el conector FMC HPC0 (P2) de la ZCU102.	34
5.2. Mapa de señales entre la placa CDHS y el conector FMC HPC0 de la ZCU102.	35
5.3. Mapa de señales entre la placa AOCS y el conector FMC HPC0 de la ZCU102.	36
5.4. Tabla NCO: frecuencias de baudrate soportadas y error medido (extracto, modo <i>full-bit</i>).	37
5.5. Presupuesto económico	40

Lista de acrónimos

AOCS *Attitude and Orbit Control System* — Subsistema de control de actitud y órbita.

ADC *Analog-to-Digital Converter* — Convertidor analógico-digital.

AXI *Advanced eXtensible Interface* — Bus de interconexión de alto rendimiento del estándar ARM AMBA.

BRAM *Block RAM* — Bloque de memoria RAM integrado en la FPGA.

BSP *Board Support Package* — Paquete de soporte de placa.

BOM *Bill of Materials* — Lista de materiales de una PCB.

CAN *Controller Area Network* — Bus de comunicaciones serie multipunto.

CDHS *Command and Data Handling System* — Subsistema de gestión de datos y telecomandos.

CMRR *Common Mode Rejection Ratio* — Razón de rechazo al modo común.

DMA *Direct Memory Access* — Acceso directo a memoria sin intervención de la CPU.

DSP *Digital Signal Processor* — Bloque de procesamiento digital de señal.

DTB *Device Tree Binary* — Árbol de dispositivos en formato binario para sistemas Linux/U-Boot.

EMI *Electromagnetic Interference* — Interferencia electromagnética.

ESD *Electrostatic Discharge* — Descarga electrostática.

ESA *European Space Agency* — Agencia Espacial Europea.

ETSIT Escuela Técnica Superior de Ingenieros de Telecomunicación.

FIFO *First In, First Out* — Estructura de datos de cola.

FMC *FPGA Mezzanine Card* — Tarjeta de expansión para FPGA según estándar VITA 57.1.

FPGA *Field Programmable Gate Array* — Matriz lógica programable en campo.

FSM *Finite State Machine* — Máquina de estados finitos.

FSBL *First Stage Boot Loader* — Cargador de arranque de primera etapa.

HDL *Hardware Description Language* — Lenguaje de descripción de hardware.

INTC *Interrupt Controller* — Controlador de interrupciones.

ISR *Interrupt Service Routine* — Rutina de servicio de interrupción.

LEO *Low Earth Orbit* — Órbita terrestre baja.

LINCE Línea de industrialización de cargas de pago y plataformas espaciales.

LVDS *Low-Voltage Differential Signaling* — Señalización diferencial de bajo voltaje.

LUT *Look-Up Table* — Tabla de búsqueda, bloque básico de una FPGA.

MMIO *Memory-Mapped I/O* — Entrada/salida con mapeado en memoria.

MPSoC *Multiprocessor System-on-Chip* — Sistema multiprocesador en chip.

NCO *Numerically Controlled Oscillator* — Oscilador controlado numéricamente.

OBC *On-Board Computer* — Ordenador de a bordo.

PCB *Printed Circuit Board* — Placa de circuito impreso.

PL *Programmable Logic* — Lógica programable (FPGA del MPSoC).

PMUFW *Power Management Unit Firmware* — Firmware de la unidad de gestión de potencia.

PS *Processing System* — Sistema de procesamiento (ARM del MPSoC).

PWM *Pulse Width Modulation* — Modulación por ancho de pulso.

RTOS *Real-Time Operating System* — Sistema operativo de tiempo real.

RTEMS *Real-Time Executive for Multiprocessor Systems* — RTOS de código abierto para sistemas empotrados.

SMD *Surface Mount Device* — Componente de montaje superficial.

SPI *Serial Peripheral Interface* — Interfaz serie para periféricos.

TCL *Tool Command Language* — Lenguaje de scripting utilizado en Vivado.

TFM Trabajo Fin de Máster.

TVS *Transient Voltage Suppressor* — Supresor de sobretensiones transitorias.

UART *Universal Asynchronous Receiver-Transmitter* — Transceptor serie asíncrono universal.

UPM Universidad Politécnica de Madrid.

VHDL *Very High Speed Integrated Circuit Hardware Description Language* — Lenguaje de descripción de hardware.

XSA *Xilinx Support Archive* — Archivo de especificación hardware exportado por Vivado para Vitis.

1. Introducción y objetivos

1.1. Introducción y motivación

El diseño de ordenadores de a bordo (OBC) para satélites exige resolver un reto de ingeniería que va más allá del procesamiento puro: conseguir que el procesador central se comunique de forma determinista y fiable con una familia heterogénea de periféricos —sensores de actitud, actuadores, cargas de pago— a través de buses serie con especificaciones eléctricas y de temporización muy diferentes entre sí. En el marco del proyecto LINCE, el grupo B105 Electronic Systems Lab de la Universidad Politécnica de Madrid asume la responsabilidad del subsistema de comunicaciones del OBC.

El trabajo se desarrolló en el laboratorio B105 en calidad de colaborador en prácticas dentro del proyecto LINCE, y la tarea inicial consistió en implementar el soporte de comunicaciones serie RS422 y RS485 sobre la plataforma de evaluación Zynq UltraScale+ ZCU102. Esta plataforma, basada en un MPSoC que integra en un mismo encapsulado un sistema de procesamiento ARM (PS) y una lógica programable tipo FPGA (PL), proporciona la flexibilidad necesaria para implementar transceptores serie a medida. Se optó por un diseño propio en VHDL en lugar de emplear IPs de terceros, con el objetivo de obtener control total sobre el comportamiento del transceptor: parámetros de temporización, gestión de errores y adaptación a los requisitos concretos del sistema.

A medida que avanzaba el desarrollo, el alcance del trabajo se fue ampliando: las tareas de hardware fueron asumiéndose progresivamente a medida que se demostró capacidad para ello. Así, además del diseño del transceptor y su driver software, se asumió también el diseño de las tarjetas de circuito impreso necesarias para validar el sistema frente a los requisitos del consorcio. La colaboración directa con Sener —socio tecnológico del proyecto LINCE, con reuniones quincenales y comunicación continua por correo— fue determinante tanto para definir esos requisitos como para iterar sobre el diseño. A su vez, Sener traslada los requisitos técnicos impuestos por Indra, que lidera el consorcio.

El resultado de todo este trabajo es una plataforma funcional que abarca el codiseño hardware-software del subsistema de comunicaciones serie, su integración bajo el sistema operativo de tiempo real RTEMS, y la fabricación y validación de dos tarjetas de prueba.

1.2. Metodología

El desarrollo de este trabajo se enmarcó en la dinámica de trabajo colaborativo del proyecto LINCE. Se mantuvieron reuniones quincenales con los ingenieros de Sener asignados al proyecto, además de comunicación continua por correo para resolver dudas técnicas y alinear requisitos. Las tareas se gestionaron mediante Jira, donde cada reunión servía

para revisar el estado de los ítems abiertos, identificar bloqueos y planificar los próximos hitos. Los requisitos de diseño de las PCBs de prueba llegaron a través de Sener, que los traslada desde Indra como líder del consorcio.

El trabajo se estructuró en dos fases principales:

Fase 1 — Familiarización y establecimiento del entorno (octubre – enero): Los primeros meses se dedicaron a establecer las bases del entorno de desarrollo: instalación y configuración de Vivado y Vitis, compilación de RTEMS 7 desde código fuente para la arquitectura AArch64 mediante el RTEMS Source Builder, y validación del flujo completo (síntesis en Vivado → empaquetado en Vitis → ejecución en RTEMS sobre la ZCU102) con un ejemplo funcional de extremo a extremo. Esta fase fue fundamental para comprender la arquitectura PS-PL del MPSoC y las particularidades del entorno de compilación cruzada para sistemas empotrados.

Fase 2 — Desarrollo e integración (febrero – junio): Con el entorno establecido, se abordó el desarrollo principal: el transceptor serie en VHDL, el driver para RTEMS, el diseño de las PCBs CDHS y AOCS, y las aplicaciones de prueba. Se siguió una metodología iterativa: cada módulo se implementaba, se validaba de forma aislada (mediante simulación en Vivado o prueba directa sobre la placa) y se integraba en el sistema global antes de avanzar al siguiente.

Quedan pendientes para la fase final, antes de la entrega del TFM en septiembre de 2026, la fabricación y validación de la PCB de diseño propio y el desarrollo de la aplicación de testing exhaustivo de los 14 transceptores en paralelo.

1.3. Objetivos

El objetivo principal de este trabajo es desarrollar una plataforma funcional y validada de comunicación con periféricos serie sobre el MPSoC Zynq UltraScale+ ZCU102, que sirva como base para el OBC del proyecto LINCE. Para alcanzarlo, se plantearon los siguientes objetivos específicos:

1. **Diseñar un IP core de transceptor serie configurable en VHDL** para la lógica programable del MPSoC, capaz de operar sobre los estándares RS422 y RS485, con parámetros configurables en tiempo de ejecución: baudrate, paridad, número de bits de datos, bits de parada y orden de bit.
2. **Desarrollar un driver de software completo en C para RTEMS** que abstraiga el acceso al hardware y permita a la aplicación gestionar hasta 14 instancias de transceptor simultáneas mediante una API orientada a interrupciones, sin recurrir a *polling* de CPU.
3. **Diseñar y fabricar las tarjetas de expansión de prueba** requeridas para validar el sistema frente a los subsistemas CDHS (gestión de datos y calentadores) y AOCS (control de actitud y órbita) del satélite, integrando interfaces RS422/RS485, CAN, SPI, SpaceWire y PWM.
4. **Desarrollar herramientas de automatización del flujo de desarrollo**, incluyendo scripts TCL para la generación automática del diseño en Vivado y scripts de despliegue de imágenes, con el fin de reducir los tiempos de iteración.

5. **Integrar y validar el sistema completo**, verificando la comunicación entre la ZCU102 y las tarjetas de expansión a través de los distintos buses implementados.

2. Marco teórico

2.1. Proyecto LINCE

El Proyecto LINCE (Línea de industrialización de cargas de pago y plataformas espaciales) es una iniciativa estratégica enmarcada en el PERTE Aeroespacial, cuyo propósito es consolidar la capacidad industrial de España en el segmento de los microsátélites (100–200 kg) destinados a órbitas bajas (LEO). Liderado por Indra, el consorcio busca desarrollar plataformas satelitales modulares, fiables y de altas prestaciones, optimizando costes y plazos mediante procesos de producción en serie. Este ecosistema integra a grandes empresas, PYMES y centros de investigación, como la Universidad Politécnica de Madrid (UPM), para habilitar servicios avanzados de telecomunicaciones, vigilancia y observación, garantizando así la autonomía estratégica de España y Europa en el sector espacial [1].

2.2. MPSoC Zynq UltraScale+ ZCU102

La placa de desarrollo sobre la que se va a trabajar en este proyecto es la Tarjeta de Evaluación de propósito general ZCU102, desarrollada por Xilinx, orientada al prototipado rápido de sistemas empujados de alta complejidad. Esta placa ofrece soporte a todas las interfaces de alta velocidad y lógica programable del MPSoC (*Multiprocessor System-on-Chip*) Zynq UltraScale+, que integra en un único encapsulado de silicio un sistema de procesamiento con cuatro núcleos y una matriz lógica programable, conectados internamente mediante buses de alto rendimiento. Esta arquitectura divide el sistema en dos dominios complementarios, permitiendo separar la carga de trabajo entre el Sistema de Procesamiento (PS, *Processing System*) y la lógica programable (PL, *Programmable Logic*).

El **PS** representa el dominio del software. Su desarrollo es análogo a la programación de cualquier microcontrolador o microprocesador clásico; está basado en una arquitectura física fija (núcleos ARM en este dispositivo) diseñada para ejecutar instrucciones de código en C o C++ de forma estrictamente secuencial. Este dominio es el encargado de ejecutar sistemas operativos (como RTEMS o Linux), gestionar la memoria y ejecutar las rutinas de control de alto nivel.

Por el contrario, la **PL** representa el dominio del hardware a medida. Se trata de una matriz tipo FPGA (*Field Programmable Gate Array*), la cual no ejecuta un programa línea a línea, sino que se reconfigura físicamente. Mediante lenguajes de descripción de hardware (como VHDL), se define la interconexión de miles de recursos internos disponibles en el silicio —tales como tablas de búsqueda (LUTs), biestables, memorias BRAM y bloques de procesamiento digital de señales (DSP)— para conformar circuitos digitales reales y

específicos. Esta característica otorga a la PL un determinismo temporal estricto y la capacidad de procesar señales con un paralelismo masivo inalcanzable para un procesador convencional.

Entre sus características destacadas, una que resulta relevante para este trabajo es la capacidad de expansión externa mediante dos conectores FMC HPC (*High Pin Count*). Estos puertos son el punto de anclaje para tarjetas de expansión o *Mezzanines*, permitiendo extraer pines digitales y pares diferenciales hacia el exterior del sistema. Siguen el estándar VITA 57.1 *FPGA Mezzanine Card (FMC) specification* [2].

2.3. Vivado y Vitis (Xilinx): PS y PL

Los programas que más se van a utilizar durante este trabajo son Vivado y Vitis, ambos desarrollados por Xilinx para el desarrollo sobre sus sistemas hardware.

2.3.1. Vivado Design Suite

Es el entorno oficial de Xilinx para el diseño, síntesis e implementación de circuitos digitales destinados a la PL. Permite programar en Lenguaje de Descripción Hardware (HDL) los circuitos que se desean implementar, ofrece una herramienta de diseño por bloques y permite realizar simulaciones temporales para verificar el comportamiento del hardware desarrollado.

En el contexto específico de este proyecto, Vivado se utiliza para encapsular los transceptores serie diseñados a medida en VHDL y enlazarlos al núcleo del procesador Zynq. Esta integración se realiza mediante el bus de interconexión de alto rendimiento AXI (*Advanced eXtensible Interface*) a través de la herramienta *Block Design*. Asimismo, Vivado gestiona el archivo de restricciones físicas (.xdc), necesario para mapear lógicamente las señales de los periféricos desde el silicio hacia los pines reales de los conectores FMC. El flujo de trabajo en Vivado culmina con la generación del *Bitstream* (el archivo binario que configura físicamente la FPGA) y la exportación de la arquitectura del hardware en un archivo unificado (.xsa).

2.3.2. Vitis Unified Software Platform

Vitis es el entorno de desarrollo integrado (IDE) proporcionado por Xilinx para la programación del dominio del software, es decir, el PS. Funciona en tándem con Vivado: importa el archivo de especificación hardware (.xsa) y genera de forma automatizada el Paquete de Soporte de la Placa o BSP (*Board Support Package*). Este BSP contiene el mapa de memoria estático y los controladores de bajo nivel (*drivers*) indispensables para que el código en C/C++ pueda acceder a los periféricos instanciados previamente en la PL.

Durante el desarrollo de este trabajo, Vitis se emplea para la compilación cruzada orientada a los núcleos ARM del sistema. Se utiliza para generar el firmware crítico de inicialización, concretamente el PMUFW (*Power Management Unit Firmware*) y el cargador de arranque de primera etapa FSBL (*First Stage Boot Loader*). Finalmente, Vitis proporciona las herramientas de empaquetado para fusionar el código ejecutable, el *Bitstream* de la PL y los archivos de arranque en un único fichero binario (BOOT.BIN), habilitando el arranque autónomo de la placa desde una memoria no volátil como una tarjeta SD.

2.4. RTEMS

RTEMS (*Real-Time Executive for Multiprocessor Systems*) es un sistema operativo de tiempo real (RTOS) de código abierto diseñado de forma específica para sistemas empujados de misión crítica. A diferencia de los sistemas operativos de propósito general, como distribuciones estándar de Linux, RTEMS no utiliza memoria virtual y opera en un único espacio de direcciones físico (arquitectura de un solo proceso y múltiples hilos). Esta arquitectura elimina la latencia de los cambios de contexto pesados, minimiza la sobrecarga del procesador y garantiza tiempos de respuesta estrictamente deterministas ante las interrupciones del hardware.

El uso de RTEMS es un estándar de facto en la industria aeroespacial europea y americana (siendo empleado activamente por agencias como la ESA y la NASA en misiones satelitales y sondas espaciales) debido a su robustez, su certificación para entornos críticos y su compatibilidad nativa con arquitecturas ARM como la del Zynq UltraScale+. En el marco del proyecto LINCE y de este TFM, RTEMS se ejecuta sobre el PS actuando como el cerebro temporal del sistema. Su función es orquestar las rutinas de control de alto nivel, gestionar la lectura/escritura de los registros de los transceptores de la PL y asegurar que las transacciones en los buses de comunicación (SpaceWire, RS422, RS485 o CAN) cumplan con los plazos de tiempo (*deadlines*) exigidos durante las pruebas *Hardware-in-the-Loop* de los subsistemas del satélite.

2.5. Comunicaciones serie

En el diseño de sistemas empujados de misión crítica y arquitecturas satelitales, las comunicaciones serie constituyen la columna vertebral para el intercambio de datos entre el Ordenador de a Bordo (OBC) y sus múltiples periféricos, tales como sensores de actitud, actuadores mecánicos y cargas de pago. Frente a las topologías de bus paralelo tradicionales, los enlaces serie proporcionan una reducción drástica de las líneas físicas necesarias, lo que se traduce de forma directa en una disminución crítica del volumen y peso del arnés del satélite. Además, al operar frecuentemente mediante señalización diferencial, mitigan en gran medida la susceptibilidad a las interferencias electromagnéticas (EMI) propias del entorno espacial.

En este apartado se detallan los fundamentos técnicos, topologías y características eléctricas de los protocolos de comunicación serie específicos que han sido seleccionados e integrados en la plataforma de hardware durante el desarrollo de este trabajo: RS485, RS422, SpaceWire y bus CAN.

2.5.1. RS485

El estándar TIA/EIA-485, comúnmente conocido como RS485, define las características eléctricas de receptores y transmisores para su uso en sistemas de comunicaciones serie balanceados (diferenciales). Su principal característica es la capacidad de operar en topologías multipunto (buses compartidos), permitiendo conectar hasta 32 transceptores estándar en el mismo par de cables (o más, si se emplean transceptores de carga fraccional).

La señalización diferencial, donde la información se transmite como la diferencia de potencial entre dos hilos trenzados (A y B), le otorga un alto Rechazo al Modo Común

(CMRR). Esto significa que cualquier ruido electromagnético inducido en el cable afecta por igual a ambas líneas, cancelándose en el receptor. Habitualmente se implementa en modo *Half-Duplex* (2 hilos), donde la transmisión y recepción comparten el medio físico, exigiendo un control estricto del flujo de datos mediante software (gestión de los pines de *Driver Enable*) para evitar colisiones. Es un estándar robusto capaz de alcanzar distancias de hasta 1.200 metros (a velocidades reducidas) o velocidades de hasta 50 Mbps en distancias cortas, requiriendo resistencias de terminación (típicamente de 120Ω) en los extremos del bus para evitar reflexiones destructivas de la señal [3, 4].

2.5.2. RS422

El estándar TIA/EIA-422 (RS422) es el predecesor técnico del RS485 y comparte con él la base física de la señalización diferencial para garantizar la inmunidad al ruido y cubrir largas distancias. Sin embargo, su principal diferencia radica en la topología de la red: el RS422 está diseñado estrictamente para comunicaciones punto a punto o punto a multipunto (conocido como *multi-drop*).

A diferencia del RS485, los transceptores RS422 no están diseñados para ceder el control del bus. En un bus RS422 solo puede existir un único transmisor (maestro) conectado a un máximo de 10 receptores (esclavos) en el mismo par de hilos. Para lograr una comunicación bidireccional continua (*Full-Duplex*), el RS422 requiere obligatoriamente el uso de cuatro hilos (dos pares trenzados: uno dedicado exclusivamente a la transmisión y otro a la recepción). Esta separación física de los canales de ida y vuelta elimina la necesidad de arbitrar el bus por software, reduciendo la sobrecarga de procesamiento y garantizando un determinismo temporal absoluto, lo que lo hace idóneo para el envío continuo de telemetría de sensores críticos [5, 6].

2.5.3. SpaceWire

SpaceWire es un estándar de red de comunicaciones espaciales de alta velocidad, coordinado por la Agencia Espacial Europea (ESA) y ampliamente adoptado a nivel mundial en misiones científicas y comerciales. Está diseñado específicamente para interconectar nodos de alto rendimiento a bordo de satélites, como memorias masivas, procesadores de instrumentación y enlaces de telemetría, ofreciendo un gran ancho de banda y una fiabilidad extrema.

A nivel físico, SpaceWire utiliza señalización diferencial de bajo voltaje (LVDS) e implementa una codificación única denominada *Data-Strobe* (DS). En lugar de transmitir una señal de reloj independiente que sufriría desvíos temporales (*skew*) respecto a los datos a altas frecuencias, la señalización DS envía los datos por un par diferencial y una señal *Strobe* por otro. El reloj se recupera en el receptor realizando una operación lógica XOR entre la señal de datos y el *Strobe*. Esto permite enlaces punto a punto *full-duplex* asíncronos con velocidades que abarcan desde los 2 Mbps hasta más de 400 Mbps. Además, su arquitectura en capas soporta el enrutamiento de paquetes, permitiendo construir topologías de red complejas mediante *routers* SpaceWire [7, 8].

2.5.4. CAN

El bus CAN es un estándar de comunicaciones serie robusto diseñado originalmente para el sector de la automoción, pero cuya alta tolerancia a fallos lo ha convertido en un estándar

de facto (*CAN for Aerospace*) para las redes internas de monitorización, telemetría y telecomandos (TM/TC) en nanosatélites y satélites comerciales.

Es un bus multipunto y multi-maestro que opera bajo el paradigma CSMA/CD+AMP (*Carrier Sense Multiple Access with Collision Detection and Arbitration on Message Priority*). En una red CAN, los nodos no tienen una dirección física; en su lugar, transmiten tramas de datos encabezadas por un identificador que define el contenido y la prioridad del mensaje. Si dos nodos intentan transmitir simultáneamente, la colisión se resuelve a nivel de bit y de forma no destructiva: el mensaje con el identificador de mayor prioridad sobrescribe al de menor prioridad sin corromper el bus, garantizando un determinismo estricto para las alarmas críticas. Además, la capa de enlace del protocolo CAN incorpora mecanismos de detección de errores por hardware (CRC, *bit stuffing*, comprobación de tramas) y confinamiento automático de nodos defectuosos, asegurando que un periférico averiado no bloquee el sistema de control de actitud u otros subsistemas vitales del Ordenador de a Bordo [9, 10].

3. Desarrollo

3.1. Preparación del entorno de desarrollo

Para llevar a cabo la implementación propuesta en este Trabajo de Fin de Máster, fue necesario establecer un entorno de desarrollo robusto que permitiera el codiseño hardware-software (PS-PL) sobre la plataforma Zynq UltraScale+ MPSoC. A continuación, se detalla la metodología seguida para la configuración de las herramientas de síntesis, la compilación del sistema operativo en tiempo real (RTEMS) y la validación del flujo de trabajo completo. El entorno de desarrollo principal se desplegó sobre una máquina virtual con una distribución Linux (Ubuntu/Debian). En este sistema se instalaron las dependencias necesarias para la compilación cruzada, incluyendo paquetes esenciales de desarrollo de C/C++, Python 3 y herramientas de control de versiones.

3.1.1. Instalación de Vivado y Vitis

Para el desarrollo de la lógica programable (PL) y la generación de imágenes de arranque, se utilizaron las herramientas oficiales de AMD Xilinx: Vivado Design Suite y Vitis Unified Software Platform. Vivado se empleó para el diseño a nivel de hardware y la generación del *bitstream*, mientras que Vitis se utilizó para empaquetar los binarios y generar la imagen de arranque (BOOT.bin).

3.1.2. Instalación de RTEMS

En lugar de utilizar binarios precompilados, se optó por compilar el sistema operativo RTEMS (versión 7) desde cero, garantizando el control total sobre la configuración del núcleo. La compilación se realizó utilizando la herramienta RTEMS Source Builder (RSB). Se generó un *toolchain* cruzado específico para la arquitectura AArch64. Se configuró y compiló el Board Support Package (BSP) correspondiente a la plataforma ZynqMP (*zynqmp_apu*), habilitando explícitamente el soporte para multiprocesamiento simétrico (SMP).

Para validar el entorno recién compilado, se ejecutaron pruebas preliminares utilizando el emulador QEMU (*qemu-system-aarch64*) configurado con la máquina *xlnx-zcu102*. Se verificó la correcta ejecución de ejemplos básicos como *hello.exe* y pruebas de rendimiento computacional como el *benchmark* Dhrystone.

3.1.3. Flujo de desarrollo completo

Una vez preparadas todas las herramientas de desarrollo, se probó a realizar un desarrollo completo de principio a fin implementando un ejemplo sencillo: una puerta AND. Para

ello, se siguieron los siguientes pasos.

Parte Vivado. Se creó un proyecto nuevo en Vivado llamado `and_gate_ps_p`, y se diseñó la puerta AND en VHDL:

Programación 3.1: AND_GATE.vhd — ejemplo de puerta AND en VHDL

```
1 library IEEE;
2 use IEEE.STD_LOGIC_1164.ALL;
3
4 entity AND_GATE is
5     Port ( A_B_IN : in  STD_LOGIC_VECTOR (1 downto 0);
6           Z_OUT  : out STD_LOGIC);
7 end AND_GATE;
8
9 architecture Behavioral of AND_GATE is
10 begin
11     Z_OUT <= A_B_IN(1) and A_B_IN(0);
12 end Behavioral;
```

Después se creó un *Block Design*, se añadió el bloque Zynq UltraScale+ MPSoC IP y se configuró automáticamente usando *Block Automation* de Vivado. Se añadió el fichero VHDL del paso anterior al *Block Design*. Se añadieron dos AXI GPIO IPs al *block design* para comunicar el PS con la PL, y se conectaron los bloques entre sí, utilizando *Automate Connection* de Vivado para todas las conexiones faltantes.

Después se creó un *Wrapper* del *Block Design* para que Vivado pudiera sintetizarlo correctamente. En la pestaña *Address Editor* pueden verse las direcciones de los registros para acceder a la parte PL desde el PS. Por último, se generó el *BitStream* y se exportó el hardware como `.xsa`.

Generación de la imagen de arranque (Vitis). Se siguieron los siguientes pasos para empaquetar el hardware generado en Vivado en un binario que el MPSoC pudiera ejecutar: se creó un nuevo componente en Vitis de tipo *Platform*, utilizando como base el `.xsa` generado; se compiló el FSBL usando el ejemplo *Zynq MP FSBL*; y se generó el archivo `BOOT.bin` mediante el generador de imágenes de Vitis con la estructura de componentes estándar (FSBL, PMUFW, bitstream de la PL, BL31, U-Boot y DTB).

Parte RTEMS. Una vez generado el archivo de arranque, se desarrolló un software de ejemplo que interactuaba con los puertos GPIO conectados por AXI, excitando todas las combinaciones posibles de la puerta AND en la PL y leyendo su salida. Fue necesario añadir un archivo de mapeado de memoria para que el sistema operativo reconociera las direcciones correspondientes a la PL, y un `wscript` para la compilación.

Para facilitar el desarrollo rápido en la parte de RTEMS, se desarrolló un script que compila la imagen de RTEMS, generando `rtems.img`. Para evitar quitar y poner la tarjeta SD constantemente, se desarrolló un script en Python que es capaz de subir la imagen de RTEMS a la tarjeta SD mediante una conexión USB.

Una vez verificado que esta metodología de desarrollo funcionaba correctamente, se estableció como el proceso estándar a seguir para los próximos desarrollos.

3.2. Diseño del transceptor serie configurable (PL)

El transceptor serie se diseñó para ser altamente configurable y adaptable a cualquier dispositivo. Los parámetros configurables son:

- Baudrate (desde 50 hasta 4.000.000 baudios).
- Número de *stop bits* (1, 1,5 o 2).
- Paridad (par, impar, marca, espacio o deshabilitada).
- Orden de los bits (LSB o MSB).
- Número de bits de datos (entre 5 y 9).

3.2.1. Transmisor

El siguiente diagrama FSM resume el comportamiento del transmisor. Debido a su tamaño, se ha dividido en tres partes para su legibilidad.

[Figura pendiente: Diagrama FSM del transmisor (tres partes)]

Figura 3.1: Diagrama FSM del transmisor serie.

Durante las pruebas sobre la placa CDHS se detectó que en las primeras recepciones aparecían ocasionalmente caracteres corruptos. Como primera hipótesis se consideró que la señal DE podría estar conmutando demasiado rápido al paso de transmisión a recepción, haciendo que el transceptor físico no tuviera tiempo de establecerse. Para corregirlo, se añadieron dos estados adicionales en la FSM del transmisor que introducen un retardo de microsegundos entre la desactivación del último bit y la desactivación de DE. Sin embargo, esto no resolvió el problema, descartando la hipótesis inicial.

3.2.2. Receptor

El siguiente diagrama FSM resume el comportamiento del receptor.

[Figura pendiente: Diagrama FSM del receptor serie]

Figura 3.2: Diagrama FSM del receptor serie.

Decisión de diseño: verificación de start-bit a mitad de período

Tras descartar el flanco de DE como causa de los caracteres corruptos, el análisis se centró en el receptor. El protocolo serie comienza con un **start-bit**: la línea, normalmente en reposo en nivel alto, cae a nivel bajo para señalar el inicio de una trama. El receptor detecta este flanco descendente y arranca la máquina de estados de recepción.

El problema era que pulsos de ruido breves en la línea generaban flancos descendentes espurios que el receptor interpretaba como *start-bits* válidos, capturando una trama de basura y enviando un carácter corrupto al *buffer*.

La solución se implementó directamente en la FSM: en el estado de recepción del *start-bit*, una vez transcurrido el **primer medio período de bit**, se vuelve a muestrear la línea antes de continuar. Si la línea ha vuelto a nivel alto, el pulso era ruido y la FSM regresa al estado IDLE sin capturar nada. Solo si la línea sigue en bajo al llegar a la mitad del bit se considera un *start-bit* legítimo y se procede con la recepción del resto de la trama.

Esta técnica —verificación a mitad de *start-bit*— es una práctica estándar de robustez en receptores UART. Su implementación en la FSM VHDL eliminó por completo la aparición de caracteres corruptos en las pruebas posteriores.

3.2.3. Oscilador controlado numéricamente (NCO)

Este oscilador sirve para generar una señal de *enable* a un baudrate seleccionado. Hay 54 valores de velocidad seleccionables almacenados en una tabla de verdad. Para poder generar frecuencias que no son divisibles exactamente de la frecuencia de reloj de la FPGA, se utiliza este circuito que corrige periódicamente el desfase generado por esta imprecisión, logrando replicar un amplio número de frecuencias con muy poco error.

El circuito consiste en un sumador acumulador: para cada frecuencia seleccionada, se tiene calculado un incremento concreto que depende del número de bits del sumador. Cuantos más bits tenga este sumador, mejor precisión se consigue. En esta implementación se han utilizado 32 bits. El incremento se suma y acumula en cada período de reloj; cuando el sumador se desborda, se utiliza el bit de *carry_out* como *tick* del NCO. Este *tick* se genera a la frecuencia deseada, y el mecanismo de acumulación garantiza que el error promedio sea inferior al de un divisor entero simple. La tabla de frecuencias soportadas y su error medido se recoge en el Anexo 1.

[Figura pendiente: Diagrama del circuito NCO de 32 bits]

Figura 3.3: Circuito del oscilador controlado numéricamente (NCO).

3.2.4. Shift-register

El registro de desplazamiento está diseñado para poder almacenar datos tanto si la configuración es *LSB-first* como *MSB-first*, de modo que una vez recibido el mensaje la palabra en la salida siempre es la correcta independientemente del modo configurado.

3.2.5. FIFO

La FIFO es un bloque IP con 9 bits de datos (para poder almacenar mensajes de 9 bits) y 512 de profundidad, que permite almacenar temporalmente varios mensajes y habilitarlos para su lectura posterior por el PS.

3.2.6. Bloque Zynq UltraScale+ MPSoC

Para que el proyecto en Vivado funcione correctamente, es importante aplicar el *preset* al bloque IP Zynq UltraScale+ antes de añadir el resto de elementos, ya que se aplican configuraciones de relojes y alimentación cruciales para el correcto funcionamiento de la PL.

3.3. Herramientas para facilitar el desarrollo del transceptor (PL)

3.3.1. Generación de transceivers con TCL

Se desarrolló un script de TCL que permite generar y conectar automáticamente un número deseado de transceptores en el *Block Design* de Vivado. Para que funcione correctamente, es necesario tener las fuentes .vhd del transceptor en el proyecto, el *Block Design* creado, y el bloque IP Zynq UltraScale+ MPSoC importado y configurado previamente; a partir de ahí, el script se encarga de añadir y conectar el resto de los bloques necesarios.

3.3.2. Generación de proyecto con TCL

También se desarrolló otro script de TCL (`regenerate_all.tcl`) que regenera el proyecto entero de Vivado con un número determinado de transceptores, partiendo desde cero. Esto permite reproducir exactamente cualquier configuración sin necesidad de guardar el proyecto de Vivado completo en el repositorio.

3.4. Generación del binario en Vitis (PS)

Una vez diseñada la lógica hardware en Vivado y exportado el hardware, se siguieron los pasos establecidos previamente para generar el binario BOOT.bin.

El proceso manual de exportación del XSA, creación de la plataforma Vitis, compilación del FSBL y ensamblado del BOOT.BIN con `bootgen` resultó tedioso de repetir en cada iteración del diseño. Para automatizarlo, se desarrolló el script `generate_boot.sh` (disponible en `tfm/04_tools/`), un *wizard* interactivo en Bash que encadena todos estos pasos de forma desatendida. El script ofrece tres puntos de entrada según el estado en que se encuentre el trabajo: partir del proyecto de Vivado y generar *bitstream* desde cero, partir de un proyecto con *bitstream* ya generado y solo exportar el XSA, o partir directamente de un XSA ya exportado. A partir de ahí invoca Vitis en modo script mediante su API Python para crear la plataforma y compilar el FSBL, y llama a `bootgen` para ensamblar el BOOT.BIN final con los componentes de arranque precompilados (PMUFW, BL31, U-Boot, DTB). Esto redujo el tiempo de iteración hardware → imagen de varios minutos de clics a una sola ejecución de terminal.

3.5. Diseño del driver serie en RTEMS (PS)

3.5.1. Motivación y contexto

El subsistema de comunicaciones del OBC requiere gestionar simultáneamente múltiples interfaces serie físicas (RS422 y RS485) desde una aplicación de tiempo real ejecutada sobre RTEMS. El bloque hardware `CONFIGURABLE_SERIAL_TOP`, implementado en VHDL en la lógica programable (PL), expone cada transceptor a través de un periférico AXI GPIO de doble canal, cuya interfaz de bajo nivel es demasiado específica para ser usada directamente desde la aplicación. Para abstraer esta complejidad y ofrecer una API uniforme e independiente de la dirección física de cada instancia, se desarrollaron los archivos `transceiver.c` y `transceiver.h`.

El diseño cubre tres requisitos fundamentales: (1) descubrimiento dinámico del hardware en tiempo de arranque, de modo que el mismo binario funcione con cualquier número de transceptores instanciados en el *bitstream*; (2) comunicación orientada a interrupciones para no bloquear el procesador durante la espera de datos; y (3) una API sencilla que permita a la aplicación enviar y recibir datos sin conocer los detalles del mapa de registros.

3.5.2. Interfaz hardware: mapa de registros

Cada instancia del transceptor ocupa una región de 4 KB del espacio de memoria del procesador, estructurada como un AXI GPIO de doble canal:

- **Canal 1 (escritura, PS → PL):** contiene el dato de transmisión (bits 8:0), los bits de control de protocolo (SEND, ERROR_OK, DATA_READ) y los campos de configuración: tasa de baudios (6 bits), bits de parada (2 bits), paridad (3 bits), bits de datos (3 bits), orden de bits y modo SLO.
- **Canal 2 (lectura, PL → PS):** contiene el byte recibido (bits 8:0) y las banderas de estado: RX_EMPTY, RX_FULL, PARITY_ERROR, FRAME_ERROR y TX_RDY.

Adicionalmente, un bloque de información del sistema situado en la dirección fija 0xA0020000 expone en tiempo de ejecución el número de transceptores instanciados, el *stride* de memoria entre instancias y la dirección base de la primera de ellas. Más allá del último transceptor se aloja el controlador de interrupciones AXI INTC, compartido por todas las instancias.

3.5.3. Arquitectura software

La librería se organiza en torno a dos estructuras de datos centrales:

`Transceiver_Config_t` agrupa los parámetros de configuración del protocolo: tasa de baudios, número de bits de datos (5 a 9), paridad (par, impar, *mark*, *space* o ninguna), bits de parada (1, 1,5 o 2), orden de bits (LSB o MSB *first*) y modo de emisión lenta (SLO, que reduce las emisiones electromagnéticas). Todos los valores se expresan mediante macros simbólicas definidas en `transceiver.h`, que corresponden directamente a los valores codificados en la LUT del NCO hardware.

`Transceiver` es el objeto *handle* que representa una instancia concreta del transceptor en ejecución. Contiene el mapa de direcciones calculado dinámicamente (`base_addr`, `intc_base` y los *offsets* derivados), las máscaras de interrupción individuales de RX y TX, y el estado software: un *buffer* circular de recepción y uno de transmisión (ambos de 4096 bytes por defecto, asignados dinámicamente), los identificadores de las primitivas RTEMS asociadas (tarea *worker*, *mutex* de RX y *mutex* de TX) y el *callback* de usuario para notificación de datos recibidos.

3.5.4. Descubrimiento dinámico del hardware

La función `Transceiver_Hardware_Discover()` lee el bloque de información del sistema al arranque y extrae el número de instancias presentes, el *stride* de memoria y la dirección base. Con estos tres valores se calcula automáticamente la dirección del INTC compartido:

$$g_intc_addr = g_hw_base + (g_hw_count \times g_hw_stride)$$

Este mecanismo permite que el mismo binario RTEMS funcione sin modificación con cualquier número de transceptores (hasta el máximo soportado de 14), cubriendo configuraciones que van desde un único canal hasta la instalación completa. La alternativa de *hardcodear* las direcciones habría requerido recompilar la aplicación para cada variante del *bitstream*.

3.5.5. Modelo de interrupciones: ISR maestra y tareas worker

El diseño de la gestión de interrupciones adopta el patrón *deferred interrupt processing* recomendado para RTEMS. Se instala una única ISR en el vector 121 (**Master_ISR**) que atiende a todos los transceptores a través del AXI INTC compartido. La ISR funciona del siguiente modo:

1. Lee el registro de interrupciones pendientes (INTC_ISR).
2. Para cada transceptor con interrupción RX activa: reconoce la interrupción (INTC_IAR), deshabilita temporalmente la línea (INTC_CIE) y envía el evento **RTEMS_EVENT_0** a la tarea *worker* correspondiente.
3. Para cada transceptor con interrupción TX activa: extrae el siguiente byte del *buffer* circular de transmisión, lo escribe en el registro de control pulsando **BIT_TX_SEND**, y limpia la interrupción. Si el *buffer* ha quedado vacío, marca **tx_busy = false**.

Cada instancia **Transceiver** tiene asociada una tarea RTEMS dedicada (**Rx_Worker_Task**) que se bloquea esperando el evento. Al recibirlo, vacía el FIFO hardware byte a byte hacia el *buffer* circular de recepción, pulsando **BIT_DATA_READ** para confirmar la lectura de cada byte al hardware. El acceso al *buffer* se protege con un semáforo binario de prioridad para evitar condiciones de carrera. Tras drenar el FIFO, la *worker* re-habilita la interrupción (INTC_SIE) e invoca el *callback* de usuario si está registrado.

La separación entre la ISR (que solo despierta la tarea) y la *worker* (que realiza el trabajo real) garantiza que el tiempo de latencia en contexto de interrupción sea mínimo y que el procesamiento se ejecute con las prioridades del planificador RTEMS.

3.5.6. Inicialización y configuración del hardware

Transceiver_Init() realiza la puesta en marcha completa de una instancia: calcula la dirección base y la dirección del INTC, asigna las máscaras de interrupción individuales (bit $2 \cdot id$ para RX y bit $2 \cdot id + 1$ para TX dentro del registro de 32 bits del INTC), asigna dinámicamente los *buffers* circulares, crea el semáforo y la tarea *worker* con **rtems_semaphore_create** y **rtems_task_create**, escribe la configuración de protocolo en el hardware y lanza la tarea *worker*.

La función **Transceiver_Global_INIT()** orquesta la inicialización completa del sistema: primero llama a **Transceiver_Hardware_Discover()** para poblar las variables globales, y a continuación a **Transceiver_Global_INTC_Init()**, que habilita el modo maestro del INTC (registro MER) e instala la **Master_ISR**.

3.5.7. API pública

La interfaz expuesta al código de aplicación se reduce a cinco funciones:

Función	Descripción
<code>Transceiver_Global_INIT()</code>	Descubrimiento hardware e inicialización del INTC.
<code>Transceiver_Init(dev, id, cfg)</code>	Inicializa la instancia <code>dev</code> con el id y configuración.
<code>Transceiver_SetRxCallback(dev, cb, arg)</code>	Registra <i>callback</i> de recepción.
<code>Transceiver_Read(dev, buf, maxlen)</code>	Extrae hasta <code>maxlen</code> bytes del buffer RX.
<code>Transceiver_SendString(dev, s)</code>	Encola la cadena <code>s</code> y arranca la transmisión.

Tabla 3.1: API pública del driver serie.

3.5.8. Decisiones de diseño destacadas

Buffer circular frente a copia directa en ISR. Dado que RTEMS no permite operaciones de copia de longitud arbitraria en contexto de interrupción sin afectar la latencia del sistema, se optó por *buffers* circulares gestionados desde la tarea *worker*. El tamaño de 4096 bytes proporciona margen suficiente para ráfagas de datos a las tasas más altas soportadas (hasta 4 Mbaudios).

Transmisión dirigida por interrupción. El primer byte de cada cadena a transmitir se escribe directamente desde `Transceiver_SendString()` para arrancar el hardware; los bytes siguientes son enviados sucesivamente por la *Master_ISR* al recibir la interrupción TX_RDY, sin necesidad de que la tarea de aplicación permanezca bloqueada durante la transmisión.

Compatibilidad multi-instancia mediante tabla global. El array `g_instances[]` permite a la *Master_ISR* localizar en $O(1)$ el objeto *Transceiver* asociado a cada par de bits de interrupción, evitando búsquedas costosas en contexto de ISR.

Modo SLO. El bit BIT_SLO del registro de control activa en el hardware VHDL una rampa de subida más lenta en las señales de salida, reduciendo el contenido espectral de alta frecuencia y mejorando la compatibilidad electromagnética en entornos con restricciones EMI.

3.6. Diseño hardware

Una vez completado el firmware del transceptor serie, se desarrolló el hardware de prueba necesario para validar el sistema en condiciones reales. El plan inicial era diseñar una única PCB propia para testear múltiples líneas serie. Posteriormente, el proyecto LINCE requirió dos tarjetas adicionales con especificaciones impuestas por Indra —la placa CDHS y la placa AOCS— para que Sener pudiera validar su firmware sobre la ZCU102. Los esquemáticos completos, BOMs y archivos de fabricación de las tres placas están disponibles en el repositorio del proyecto bajo `HARDWARE/`.

3.6.1. Diseño de la placa de comunicación serie (diseño propio)

El objetivo de esta placa es poder ejercitar simultáneamente los 14 transceptores del IP core, replicando las topologías de bus reales que se encontrarán en el satélite: RS485 multipunto con múltiples nodos en el mismo cable, y RS422 en configuración *master/slave* con cruce de TX y RX. A diferencia de las placas CDHS y AOCS, cuyas especificaciones

vinieron dictadas por Indra, esta placa fue diseñada con total libertad para maximizar la flexibilidad de los ensayos en laboratorio.

La arquitectura general de la placa divide los 14 canales en dos grupos conectados al FMC:

- **7 drivers RS485 (RS0–RS6)** conectados a un bus diferencial común mediante *jumpers*, formando una topología multipunto configurable.
- **7 drivers RS422 (RS7–RS13)** organizados en dos buses *master/slave* independientes (BUS 1: 1 *master* + 3 *slaves*; BUS 2: 1 *master* + 2 *slaves*), con la línea TX cruzada con RX para emular la conexión real punto a punto RS422.

[Figura pendiente: arquitectura general de la placa — 7 drivers RS485, conector FMC, 7 drivers RS422]

Figura 3.4: Arquitectura general de la placa de comunicación serie.

Topología RS485: bus compartido configurable por jumper

La decisión de diseño más relevante de esta placa es el mecanismo de bus compartido RS485 sin necesidad de recablear. Cada driver RS485 tiene un conector Dupont de 3 pines (A+, B–, GND) que actúa a la vez como punto de conexión hacia el exterior y como punto de interconexión entre drivers adyacentes. Colocando un *jumper* entre dos conectores consecutivos, las líneas A y B de ambos drivers quedan físicamente unidas, formando un segmento de bus RS485 compartido. Retirando el *jumper*, el driver queda independiente y puede conectarse a un periférico externo de forma individual.

Este mecanismo permite configurar por hardware cualquier partición de los 7 drivers RS485 en uno o varios buses, sin modificar el firmware ni el cableado exterior.

Selector de conector Micro-D para RS485

Los conectores de mayor densidad (Micro-D) de la placa son compartidos entre el Driver 0 y el Driver 6 mediante un *jumper* de selección de hardware. Dependiendo de la posición del *jumper*, los conectores Micro-D quedan asignados a uno u otro driver, lo que permite usar un único tipo de conector con cableado de satélite sin duplicar el número de conectores en la placa.

Topología RS422: master/slave configurable por jumper

Para RS422, cada driver *slave* dispone de un conector con las líneas TX cruzadas a RX (TX-Y+/TX-Z– conectadas a RX del bus), replicando la conexión estándar punto a punto RS422 donde el receptor del esclavo escucha las transmisiones del *master*. Al igual que en RS485, un *jumper* determina si el driver se une al bus común o permanece independiente.

El esquemático completo (12 hojas) y la BOM están disponibles en `HARDWARE/lince_comunicacion_serio` del repositorio del proyecto.

3.6.2. Diseño de la placa CDHS

La placa LINCE3 CDHS BreakoutBox es una tarjeta de interconexión entre la ZCU102 (conectada por FMC) y los periféricos del subsistema de gestión de datos y calentadores del satélite. Sus especificaciones fueron definidas por Indra: 6 conectores D-Sub-9 (J1–J6), interfaces CAN redundante, tres canales RS422/RS485, cuatro líneas PWM para calentadores y un ADC para termistores.

[Figura pendiente: diagrama jerárquico CDHS — bloque CAN (J1), RS (J2–J4), PWM (J5), ADC/THERM (J6)]

Figura 3.5: Diagrama de bloques de la placa CDHS.

[Figura pendiente: render 3D de la PCB CDHS]

Figura 3.6: Render 3D de la placa CDHS.

Decisión de nivel de tensión: 1,8 V

Todos los bancos de pines del conector FMC HPC utilizados por esta placa operan a 1,8 V. Se buscaron deliberadamente componentes con VIO nativo a 1,8 V para evitar etapas de adaptación de nivel adicionales entre el MPSoC y los transceptores. Esta decisión simplifica el diseño y reduce el número de componentes activos en la cadena de señal.

Subsistema CAN

El bus CAN implementa topología redundante con dos instancias independientes: CAN Nominal (CAN_NOM) y CAN Redundante (CAN_RED). Se seleccionó el transceptor **TCAN1044AVDRQ1** de Texas Instruments por su compatibilidad nativa con VIO de 1,8 V, grado automotriz y baja corriente en modo *standby*. Ambos canales convergen en el conector J1.

Las resistencias de terminación siguen el esquema *split termination* recomendado en las notas de aplicación del estándar CAN (ISO 11898-2). En lugar de una única resistencia de 120 Ω entre CANH y CANL, se emplean **dos resistencias de 60 Ω en serie** con un **condensador de 4,7 nF** conectado desde el punto medio al plano de masa. Cada una de las dos resistencias de 60 Ω está controlada por un *jumper* independiente, lo que permite habilitar o deshabilitar la terminación sin modificar el circuito. La resistencia equivalente sigue siendo 120 Ω cuando ambos *jumpers* están colocados, y el condensador de derivación crea un camino de baja impedancia para las interferencias de modo común de alta frecuencia hacia masa, mejorando el comportamiento EMC del bus.

Las líneas CANH y CANL de cada canal incorporan diodos de protección ESD **ESDCAN24-2BLY**, específicamente diseñados para buses CAN, que clampean transitorios por encima de su tensión de ruptura sin introducir una capacitancia parásita significativa.

[Figura pendiente: esquemático CAN.SchDoc — transceptores TCAN1044 con terminación split y diodos ESD]

Figura 3.7: Subsistema CAN de la placa CDHS.

Subsistema RS422/RS485

Se disponen tres canales serie idénticos e independientes (RS1, RS2, RS3), cada uno con el transceptor **THVD1424RGTR**. Este integrado fue elegido porque soporta VIO de 1,8 V, incorpora resistencias de terminación internas conmutables y permite seleccionar entre modo *Half-Duplex* y *Full-Duplex* sin cambiar el hardware. La configuración se expone mediante *jumpers* físicos de 2 pines: **H/F** (conmuta entre *Half-Duplex* y *Full-Duplex*), **SLR** (habilita control de *slew rate*), **TERM_TX / TERM_RX** (conectan las resistencias de terminación internas). Los tres canales salen por los conectores J2, J3 y J4.

Tanto las líneas RX como las TX de cada canal llevan un par de diodos TVS **SM712-02HTG** conectados entre las líneas diferenciales y el plano de masa.

Decisión de diseño: cortocircuito DE–RE. El THVD1424 expone dos pines de control independientes: **DE** (*Driver Enable*, activo en alto) habilita el transmisor, y **RE** (*Receiver Enable*, activo en bajo) habilita el receptor. En el esquemático, ambos pines están conectados a la misma señal de control procedente del MPSoC, de modo que una única línea digital controla simultáneamente transmisor y receptor de forma complementaria (tabla 3.2).

Señal DE/RE	Transmisor (DE)	Receptor (RE activo bajo)
1 (alto)	Habilitado	Deshabilitado
0 (bajo)	Deshabilitado	Habilitado

Tabla 3.2: Control complementario DE/RE en el THVD1424.

Esta decisión responde a dos motivos. Por un lado, el equipo de Indra comunicó que en la arquitectura del satélite los esclavos solo transmiten bajo demanda del OBC, por lo que no existe un escenario real en el que un nodo deba transmitir y recibir simultáneamente. Por otro lado, en modo *Half-Duplex* las líneas TX y RX comparten físicamente el mismo par diferencial A/B; si el receptor permaneciera activo durante la transmisión, el nodo escucharía su propio eco.

[Figura pendiente: RS.SchDoc — circuito completo de un canal RS con THVD1424RGTR y diodos TVS]

Figura 3.8: Canal RS de la placa CDHS con THVD1424RGTR.

Subsistema PWM (calentadores)

Cuatro señales PWM de 1,8 V procedentes del FMC se elevan a 3,3 V mediante el adaptador de niveles **TXU0104PWR** para excitar los circuitos de control de calentadores

externos. Las cuatro líneas adaptadas salen por J5.

Para la validación de esta interfaz se diseñó un bloque VHDL autónomo, `PWMx4_auto_test`, sintetizado en la PL junto al resto del diseño de Vivado. El bloque genera cuatro señales PWM independientes con *duty cycle* del 50 % a partir del reloj de 100 MHz de la FPGA: canal 0 a 10 kHz, canal 1 a 5 kHz, canal 2 a 1 kHz y canal 3 a 100 Hz. Al ser completamente autónomo no requiere ningún driver ni intervención del PS, lo que permitió verificar el subsistema PWM de la placa con el mismo BOOT.BIN utilizado para las pruebas serie.

Subsistema ADC (termistores)

Cuatro entradas analógicas de termistores (CH0–CH3) se digitalizan con el ADC SAR de 12 bits **ADS7950QDBTRQ1**, comunicado al procesador por SPI a 1,8 V. La circuitería analógica opera a 3,3 V; ambos carriles (+VBD a 1,8 V y +VA a 3,3 V) son independientes para evitar acoplos entre el dominio digital y el analógico [11]. Los termistores se conectan por J6.

Alimentación

El conector FMC suministra 12 V, 3,3 V y el carril VADJ a 1,8 V. El convertidor conmutado **R-78E5.0-1.0** genera los 5 V necesarios para la etapa de potencia de los transceptores RS y CAN a partir de los 12 V del FMC. Se eligió este módulo DC/DC por su integración en un *footprint* reducido de 3 pines (equivalente a un regulador lineal) y su eficiencia $\geq 96\%$, evitando la disipación térmica que tendría un LDO con ese diferencial de tensión.

Conectores externos

Los seis puertos D-Sub-9 (J1–J6) son de tipo macho o hembra según la convención de uso. Los escudos metálicos de todos los conectores están conectados al plano de GND para mantener la continuidad de apantallamiento EMI con los cables blindados.

El mapa completo de señales entre la placa CDHS y el conector FMC se encuentra en el Anexo 1 de este documento. El esquemático completo y la BOM están disponibles en `HARDWARE/CDHS/`.

3.6.3. Diseño de la placa AOCS

La placa LINCE3 AOCS BreakoutBox es la tarjeta de interconexión para el subsistema de control de actitud y órbita del satélite. Al igual que la CDHS, conecta la ZCU102 mediante FMC y expone las interfaces al exterior a través de 8 conectores D-Sub-9 (J1–J8).

[Figura pendiente: diagrama jerárquico AOCS — bloques RS (J1, J3–J5, J8), MOT-PWM (J2), SpaceWire (J6, J7)]

Figura 3.9: Diagrama de bloques de la placa AOCS.

[Figura pendiente: render 3D de la PCB AOCS]

Figura 3.10: Render 3D de la placa AOCS.

Canales RS422/RS485

La placa incorpora 5 canales serie (RS1, RS3, RS4, RS5, RS8) con exactamente el mismo diseño de transceptor THVD1424RGTR descrito en la sección de la placa CDHS, incluyendo el cortocircuito DE-RE y los *jumpers* de configuración H/F, SLR y terminación.

Canales SpaceWire

Se implementan dos enlaces SpaceWire *full-duplex* independientes (SPW1 y SPW2) mediante señalización diferencial LVDS, sin transceptor activo adicional, ya que el estándar SpaceWire opera directamente a niveles LVDS compatibles con los bancos del FMC. Los pares diferenciales —cuatro por enlace: DIN, SIN, SOUT y DOUT— se han trazado en la PCB con técnicas de igualación de longitud (*length matching*) para minimizar el *skew* entre el par de datos y el par de *strobe*, y con impedancia característica controlada de 100 Ω diferencial.

[Figura pendiente: esquemático SpaceWire — cuatro pares diferenciales y su conexión al FMC]

Figura 3.11: Subsistema SpaceWire de la placa AOCS.

Control de motores (MOT-PWM)

Seis señales PWM de 1,8 V procedentes del FMC se elevan al nivel de tensión requerido por los puentes en H externos mediante un adaptador de niveles. Las señales se agrupan en tres pares (X, Y, Z para los tres ejes del satélite), con dos líneas por eje para controlar la dirección de giro del motor. Salen por J2. La alimentación de potencia de los motores se conecta mediante dos bornes banana independientes de la alimentación de la placa.

Alimentación

Misma arquitectura que la placa CDHS: 12 V del FMC, convertidor DC/DC para los 5 V de potencia, y VADJ 1,8 V para los transceptores.

El mapa completo de señales entre la placa AOCS y el conector FMC se encuentra en el Anexo 1. El esquemático completo y la BOM están disponibles en [HARDWARE/AOCS/](#).

3.6.4. Fabricación

Una vez validados los esquemáticos y completado el rutado de las PCBs, se encargaron las placas con stencil a **PCBWay**, y los componentes a **Mouser** y **DigiKey** según disponibilidad. El proceso de montaje en el laboratorio siguió los pasos habituales de soldadura por reflujo SMD:

1. **Aplicación de pasta de soldadura.** Se fijó la placa a la mesa con placas de idéntico grosor alrededor y cinta de pintor. Se colocó el stencil alineado y se extendió la pasta con espátula.
2. **Colocación de componentes.** Con pinzas de punta fina, se colocaron los componentes SMD sobre la pasta. Se montaron dos placas en paralelo para optimizar el tiempo de proceso.

3. **Reflujo.** Las placas se introdujeron en el horno de reflujo del laboratorio seleccionando el perfil de temperatura adecuado para la pasta de soldadura utilizada.

[Figura pendiente: foto del proceso de fabricación — aplicación de pasta con stencil]

Figura 3.12: Aplicación de pasta de soldadura con stencil.

[Figura pendiente: foto de la placa CDHS soldada (cara superior)]

Figura 3.13: Placa CDHS tras el proceso de soldadura.

[Figura pendiente: foto de la placa AOCS soldada (cara superior)]

Figura 3.14: Placa AOCS tras el proceso de soldadura.

3.7. Desarrollo de herramientas de testing

3.7.1. Aplicación de testing del driver serie en RTEMS

La aplicación de testing del driver serie está estructurada en tres archivos:

- `init.c`: configuración del sistema RTEMS mediante macros de `confdefs.h`.
- `transceiver.c` + `transceiver.h`: driver del transceptor serie sobre AXI GPIO.
- `main.c`: aplicación de testing que usa la API del driver.

Configuración del sistema RTEMS — `init.c`

`init.c` es el fichero de configuración estática de RTEMS. No contiene lógica de aplicación; define el entorno de ejecución mediante macros que `<rtems/confdefs.h>` convierte en estructuras de datos internas del kernel:

Programación 3.2: Configuración estática de RTEMS en `init.c`

```

1  CONFIGURE_APPLICATION_NEEDS_CLOCK_DRIVER    //
    rtems_task_wake_after()
2  CONFIGURE_APPLICATION_NEEDS_CONSOLE_DRIVER  // stdin/stdout (
    printf, fgets)
3  CONFIGURE_MICROSECONDS_PER_TICK 10000      // Tick = 10 ms
    (100 Hz)
4  CONFIGURE_UNLIMITED_OBJECTS                // Sin limite de
    tareas/semaforos
5  CONFIGURE_UNIFIED_WORK_AREAS               // Un unico pool de
    memoria
6  CONFIGURE_INIT_TASK_STACK_SIZE (64 KB)     // Stack grande para
    Init

```

Aplicación de testing — main.c

La aplicación tiene tres secciones claramente separadas:

Callback de recepción `on_rx_data()`. Se registra en el driver y se invoca desde la tarea *worker* cuando llegan datos. Implementa un ensamblador de líneas por canal: por cada byte recibido, acumula hasta encontrar un carácter de fin de línea (`\n` o `\r`), momento en el que imprime la línea completa con el formato `[RX UART 02]: mensaje`. Hay un `AppLineBuffer` (1024 bytes) por cada uno de los 14 canales, declarados estáticamente en el array `rx_lines[MAX_TRANSCEIVERS]`, para evitar mezclar caracteres de UARTs distintas en la misma línea de consola.

Tarea de consola TX `Tx_Console_Task()`. Es una tarea RTEMS independiente (prioridad 100, baja) que bloquea en `fgets(stdin)` esperando comandos del usuario por la consola USB. El protocolo de comandos es:

Programación 3.3: Protocolo de comandos de la aplicación de testing

1	<ID> <MENSAJE>	-> Envía MENSAJE por la UART numero ID
2	ALL <MENSAJE>	-> Envía MENSAJE por TODAS las UARTs
3	<ID> SLO ON limitado	-> Reconfigura UART ID con Slew Rate limitado
4	<ID> SLO OFF	-> Reconfigura UART ID con Slew Rate normal
5	ALL SLO ON/OFF UARTs	-> Aplica configuracion SLO a todas las UARTs

Los comandos SLO re-ejecutan `Transceiver_Init()` con una configuración distinta, lo que permite reconfigurar el hardware en caliente sin reiniciar el sistema.

Función `Init()` — punto de entrada RTEMS. RTEMS la llama al arrancar en lugar de `main()`. La secuencia de inicialización es:

1. `mmu_map_pl_axi_early()`: mapea en la MMU la región del PL para acceso *bare-metal*.
2. `Transceiver_Global_INIT()`: descubre el hardware e instala la ISR maestra.
3. Bucle de inicialización: `Transceiver_Init()`, `Transceiver_SetRxCallback()` y `Transceiver_SerN Lista.\r\n")` para cada UART.
4. Creación y lanzamiento de `Tx_Console_Task`.
5. `rtems_task_delete(RTEMS_SELF)`: la tarea Init se elimina a sí misma; el kernel queda con las *worker tasks* + la tarea de consola.

3.7.2. Aplicación de testing de las placas CDHS y AOCS

Ampliación en PL para soportar interfaces adicionales CDHS

Para poder probar la funcionalidad de la placa CDHS, fue necesario añadir soporte firmware que controlara las líneas adicionales a RS422 y RS485. La configuración del entorno fue la siguiente: se preparó el hardware en la PL para probar las líneas de RS422/RS485, SPI, CAN y PWM creando un proyecto en Vivado con el script TCL `regenerate_all` con 3 transceptores serie, y añadiendo el bloque de control PWM. Además se configuró

el PS para conectar SPI y CAN con el exterior activando las líneas de SPI 0, CAN 0 y CAN 1 como interfaces externas hacia el conector FMC.

Para controlar las líneas de PWM se instanció el bloque `PWMx4_auto_test`, que genera desde la PL cuatro señales PWM a 10 kHz, 5 kHz, 1 kHz y 100 Hz con *duty cycle* del 50 %, sin necesidad de ningún driver en el PS. Se utilizó el mismo `BOOT.bin` para todas las pruebas de CDHS.

Para probar el ADC ADS7950 de la placa CDHS, se desarrolló una pequeña aplicación de lectura SPI sobre RTEMS. Dado que el BSP de RTEMS 7 para ZCU102 no incluía en ese momento un driver SPI de alto nivel, se accedió directamente al controlador SPI Cadence integrado en el PS mediante mapeado de registros en memoria (MMIO), siguiendo el mapa de registros descrito en el Manual de Referencia Técnico del Zynq UltraScale+ [12]. El protocolo de comunicación con el ADC —formato de trama de 16 bits, selección de canal en *Manual Mode* y gestión de la latencia de conversión— se implementó conforme al *datasheet* del ADS7950 [11].

Para probar el CAN, se utilizó una aplicación desarrollada por Diego Ramos, compañero en el laboratorio B105 y en el proyecto LINCE. En su aplicación, se configuran las líneas de CAN y se establecen tests que comprueban que la conexión por CAN funciona correctamente.

Ampliación para soportar interfaces adicionales AOCS

Para probar la placa AOCS, se generó un proyecto de Vivado con `regenerate_all` con 5 transceptores, añadiendo un bloque VHDL para controlar los puentes en H por PWM (`Motor_H_bridge_test.vhd`). Este bloque genera 3 PWMs y los enruta por la línea 1 o línea 2 de cada motor en función de la dirección deseada. Para estas pruebas, se fijó un tiempo para cada dirección de manera que durante unos segundos el PWM fuera en un sentido (línea 1 activa, línea 2 a 0) y durante los siguientes segundos en el sentido contrario.

La comunicación por SpaceWire no se llegó a probar por falta de tiempo al adquirir los arneses Micro-D9.

3.7.3. Validación de hardware: arneses y equipamiento

Arnés para verificar comunicaciones RS422

Arnés a 4 hilos, cruzando las líneas TX-P/N de un driver con las RX-P/N del opuesto.

Arnés para verificar comunicaciones RS485

Arnés a 2 hilos, en el que se conectan A+ y B− de un driver con los del otro.

Arnés para verificar comunicaciones CAN

Se conectan las líneas CAN_H y CAN_L del canal nominal con las del canal redundante.

Equipo de laboratorio utilizado

Se utilizó un osciloscopio y una fuente de alimentación de hasta 32 V.

4. Resultados

En este capítulo se recogen los resultados de la validación hardware del sistema completo. Las pruebas se realizaron con la placa CDHS y la placa AOCS conectadas a la ZCU102 mediante FMC. La placa de comunicación serie de diseño propio queda pendiente de fabricación y sus resultados se incorporarán antes de la entrega final.

4.1. Montaje del sistema

[Figura pendiente: foto del sistema completo montado — ZCU102 con placa CDHS en FMC J5]

Figura 4.1: Sistema completo montado: ZCU102 con la placa CDHS conectada al FMC.

[Figura pendiente: foto de la placa CDHS soldada — vista superior]

Figura 4.2: Placa CDHS con componentes montados (vista superior).

[Figura pendiente: foto de la placa AOCS soldada — vista superior]

Figura 4.3: Placa AOCS con componentes montados (vista superior).

4.2. Validación de comunicaciones serie RS422/RS485

Las pruebas de comunicaciones serie se realizaron con la aplicación de testing en RTEMS, que expone una consola interactiva por USB donde el usuario puede enviar mensajes a cualquier transceptor con el formato <ID><MENSAJE> o a todos simultáneamente con ALL <MENSAJE>. Se construyeron arneses de *loopback* para interconectar los canales:

- **RS422:** arnés a 4 hilos cruzando TX_{P/N} de un canal con RX_{P/N} del opuesto.
- **RS485:** arnés a 2 hilos conectando A+ y B− de un canal con A+ y B− del otro.

4.2.1. CDHS — 3 transceptores

Al arrancar, el driver descubrió automáticamente los 3 transceptores instanciados en el *bitstream* e imprimió sus direcciones base (0xA0000000, 0xA0001000, 0xA0002000), confirmando el mecanismo de descubrimiento dinámico de hardware. A continuación, la consola reportó UART 00 [OK], UART 01 [OK], UART 02 [OK].

Las pruebas de *loopback* mostraron que los mensajes enviados por UART 0 llegaban a UART 2 y viceversa, y los enviados por UART 1 llegaban correctamente. En las primeras recepciones de esta captura aparecen caracteres corruptos (?), correspondientes a una iteración previa del diseño antes de aplicar la corrección de robustez del receptor descrita en el Capítulo 3 (verificación de *start-bit* a mitad de período). Una vez implementada esa mejora en la FSM del receptor, los caracteres corruptos desaparecieron por completo en las pruebas posteriores.

Programación 4.1: Salida de terminal — prueba RS422/RS485 con la placa CDHS (3 transceptores)

```

1 [TRANSCIEVER DEBUG] Detectados: 3 | Base: 0xA0000000 | INT: 0
   xA0003000
2 UART 00 [OK]  UART 01 [OK]  UART 02 [OK]
3 CMD> 0 HOLA MUNDO
4 Tx -> UART 0: OK
5 [RX UART 02]: HOLA MUNDO
6 CMD> 2 HOLA MUNDO
7 Tx -> UART 2: OK
8 [RX UART 00]: HOLA MUNDO

```

[Figura pendiente: captura completa del terminal *cdhs_testing_rs* disponible en *tfm/terminal/*]

Figura 4.4: Captura completa del terminal de pruebas RS en la placa CDHS.

4.2.2. AOCS — 5 transceptores

Con el *bitstream* de 5 transceptores para la placa AOCS, el descubrimiento detectó correctamente 5 instancias (0xA0000000–0xA0004000, INTC en 0xA0005000). Las pruebas de *loopback* entre distintos pares de canales —UART 0↔4, UART 0↔3, UART 0↔2, UART 0↔1— resultaron todas correctas sin errores de *framing*.

Programación 4.2: Salida de terminal — prueba RS422/RS485 con la placa AOCS (5 transceptores)

```

1 [TRANSCIEVER DEBUG] Detectados: 5 | Base: 0xA0000000 | INT: 0
   xA0005000
2 CMD> 0 HOLA -> [RX UART 04]: HOLA
3 CMD> 4 HOLA -> [RX UART 00]: HOLA
4 CMD> 0 HOLA -> [RX UART 03]: HOLA
5 CMD> 0 HOLA -> [RX UART 01]: HOLA

```


[Figura pendiente: captura completa del terminal `aocs_testing_rs` disponible en `tfm/terminal/`]

Figura 4.5: Captura completa del terminal de pruebas RS en la placa AOCS.

4.2.3. Prueba con 13 transceptores simultáneos

Para verificar el correcto funcionamiento del driver con un número elevado de instancias, se cargó un *bitstream* con 13 transceptores instanciados, conectando los canales adicionales a los pines externos del conector J3 de la ZCU102. El sistema arrancó y operó correctamente con todos los canales, confirmando que la arquitectura de ISR maestra con tareas *worker* individuales escala sin problemas hasta el número máximo de instancias soportado. La prueba con los 14 transceptores sobre la placa de comunicación serie de diseño propio queda pendiente de fabricación.

4.3. Validación del bus CAN

Las pruebas del bus CAN se realizaron con la aplicación de test desarrollada por Diego Ramos (compañero del laboratorio B105 en el proyecto LINCE), que configura los canales CAN y ejecuta una batería de tests de enlace.

La suite de tests ejecutó 26 pruebas cubriendo inicialización, *loopback* interno, transferencia física CAN0↔CAN1, filtrado de identificadores estándar y extendido, manejo de tramas RTR y pruebas de interrupción por hardware. El resultado fue **26/26 tests PASS, 0 FAILED**:

Programación 4.3: Resumen de resultados del test CAN (placa CDHS)

```

1 [PASS] CAN0 initialization (Fast Mode)
2 [PASS] CAN1 initialization (Slow Mode)
3 [PASS] Loopback RX ID matches TX ID
4 [PASS] Loopback RX Data matches TX Data
5 [PASS] CAN0 received correct ID from CAN1      (fisico)
6 [PASS] CAN0 received correct Data from CAN1    (fisico)
7 [PASS] Standard ID: Exact Match Accepted (0x1A4)
8 [PASS] Extended ID: Exact Match Accepted (0x12345678)
9 [PASS] RTR Filter: Accepted matching Remote Request
10 [PASS] Tx0k fired successfully
11 [PASS] Rx0k fired and FIFO was drained successfully (No Storm)
12 ... (26/26 PASS, 0 FAILED)

```

Un resultado relevante adicional fue la verificación del efecto de las resistencias de terminación sobre la integridad de la señal CAN. Sin terminación, la señal presentaba reflexiones visibles que degradaban los flancos; con ambas resistencias de 60 Ω conectadas, la forma de onda resultó limpia y bien definida.

[Figura pendiente: captura de osciloscopio — señal CAN sin resistencias de terminación]

Figura 4.6: Señal diferencial CAN sin resistencias de terminación.

[Figura pendiente: captura de osciloscopio — señal CAN con resistencias de terminación]

Figura 4.7: Señal diferencial CAN con resistencias de terminación ($60\ \Omega + 60\ \Omega$).

4.4. Validación del ADC SPI (termistores CDHS)

La aplicación de lectura del ADC ADS7950 muestreó los cuatro canales a 500 ms de intervalo e imprimió los valores digitales por el terminal. Para verificar la linealidad de la cadena de adquisición se aplicaron tensiones de referencia conocidas a las entradas analógicas:

- Tensión de entrada 0 V $\rightarrow$ valor digital próximo a 0 (fondo de escala inferior).
- Tensión de entrada 3,3 V $\rightarrow$ valor digital próximo a 4095 (fondo de escala superior, 12 bits).

Los resultados confirmaron el rango de conversión esperado, validando el driver SPI de bajo nivel y la cadena analógica de la placa CDHS. La captura registra el canal CH2 siendo sometido a un barrido de tensión con la fuente de laboratorio:

Programación 4.4: Lectura del ADC ADS7950 durante barrido de tensión en CH2

```

1 ADS7950: CH0= 244  CH1=  43  CH2=   0  CH3=  19  <-  tension en
   CH2 = 0 V
2 ADS7950: CH0= 245  CH1=  43  CH2=1577  CH3=  19  <-  subida
3 ADS7950: CH0= 245  CH1=  43  CH2=3419  CH3=  21  <-  cerca del
   maximo
4 ADS7950: CH0= 245  CH1=  43  CH2=3565  CH3=  21  <-  ~2.9 V
   aplicados
5 ADS7950: CH0= 245  CH1=  43  CH2=2436  CH3=  22  <-  bajada
6 ADS7950: CH0= 246  CH1=  43  CH2= 102  CH3=  22  <-  de vuelta
   a 0 V

```

Los canales CH0 (≈ 245), CH1 (≈ 43) y CH3 (≈ 19) mostraron valores estables bajos durante todo el ensayo, correspondientes a las entradas no excitadas.

[Figura pendiente: captura completa del terminal `cdhs_testing_adc.txt` disponible en `tfm/terminal/`]

Figura 4.8: Captura completa del terminal de lectura ADC (placa CDHS).

4.5. Validación PWM (calentadores CDHS y puentes en H AOCS)

Las señales PWM generadas por el bloque `PWMx4_auto_test` se midieron con el osciloscopio sobre el conector J5 de la placa CDHS, verificando los cuatro canales: 10 kHz, 5 kHz, 1 kHz y 100 Hz, todos con *duty cycle* del 50 %.

Para la placa AOCS, el bloque VHDL de control de motores generó señales PWM alternando la dirección de giro cada pocos segundos (línea 1 activa con línea 2 a 0, y a continuación línea 1 a 0 con línea 2 activa), permitiendo verificar el control de puentes en H con una fuente de alimentación externa conectada a los bornes banana.

[Figura pendiente: captura de osciloscopio — señal PWM en J5 de la placa CDHS (frecuencia y duty cycle)]

Figura 4.9: Señal PWM medida en el conector J5 de la placa CDHS.

[Figura pendiente: captura de osciloscopio — par de señales PWM complementarias de control de motor (AOCS)]

Figura 4.10: Señales PWM de control de motor en la placa AOCS (par complementario para puente en H).

5. Conclusiones y líneas futuras

5.1. Conclusiones

El objetivo principal de este trabajo era desarrollar una plataforma funcional de comunicación con periféricos serie sobre el MPSoC Zynq UltraScale+ ZCU102, adaptada a los requisitos del proyecto LINCE. A lo largo del desarrollo se han obtenido los siguientes resultados:

El transceptor serie configurable diseñado en VHDL funciona correctamente sobre la lógica programable del ZCU102. Soporta los estándares RS422 y RS485 con configuración completa de protocolo en tiempo de ejecución, y el NCO de 32 bits implementado garantiza un error de baudrate inferior a 20 ppm para la mayoría de las velocidades estándar validadas, desde 9.600 hasta 4.000.000 baudios.

El driver desarrollado para RTEMS abstrae el hardware de forma efectiva y permite gestionar hasta 14 instancias simultáneas del transceptor mediante una arquitectura orientada a interrupciones. La separación entre la ISR maestra y las tareas *worker* garantiza tiempos de latencia mínimos en la atención de interrupciones, manteniendo el determinismo propio de un sistema de tiempo real.

Las dos tarjetas de circuito impreso diseñadas y fabricadas —la placa CDHS y la placa AOCS— cumplen con las especificaciones impuestas por Indra a través de Sener y han permitido validar el sistema de comunicaciones en un entorno hardware real. La fabricación se realizó en el propio laboratorio mediante proceso de soldadura por reflujo con stencil.

La validación experimental ha confirmado el correcto funcionamiento de las interfaces RS422, RS485 y CAN de la placa CDHS, así como de las interfaces RS422 y RS485 de la placa AOCS. En el caso del bus CAN, se comprobó experimentalmente la importancia de las resistencias de terminación para la integridad de la señal diferencial. El driver SPI del ADC ADS7950 produjo lecturas coherentes con la tensión de entrada aplicada, validando la cadena de adquisición analógica de la placa CDHS.

En conjunto, el trabajo ha servido como contribución directa al proyecto LINCE, proporcionando al laboratorio B105 una base funcional y documentada para el subsistema de comunicaciones del ordenador de a bordo del satélite.

5.2. Líneas futuras

A partir del trabajo realizado, se identifican las siguientes líneas de continuación:

Validación SpaceWire. La interfaz SpaceWire de la placa AOCS no pudo probarse por falta del arnés Micro-D9 adecuado. Completar esta validación es el paso inmediato más relevante, ya que SpaceWire es el protocolo de alta velocidad previsto para los enlaces de mayor ancho de banda del satélite.

Fabricación y validación de la PCB de diseño propio. La primera placa diseñada, orientada a la prueba simultánea de múltiples líneas serie con posibilidad de interconexión en buses compartidos, está pendiente de fabricación. Su validación completaría el conjunto de hardware de prueba.

Aplicación de testing exhaustivo de los 14 transceptores. El driver soporta hasta 14 instancias simultáneas pero la aplicación de testing actual no las ejerce todas a la vez de forma controlada. Desarrollar una aplicación que pruebe los 14 canales en paralelo, con métricas de *throughput* y tasa de errores, permitiría caracterizar completamente el rendimiento del sistema.

Migración a AXI-Stream con transferencia por DMA. La arquitectura actual del transceptor serie utiliza AXI GPIO como interfaz entre el PS y la PL, lo que implica que la CPU interviene en cada byte transferido. Durante el desarrollo se identificó una alternativa significativamente más eficiente: reemplazar AXI GPIO por **AXI-Stream** para el flujo de datos entre PS y PL, y añadir un controlador **DMA** (*Direct Memory Access*) que mueva los bloques de datos directamente entre la memoria del procesador y la PL sin intervención de la CPU. Esto permitiría que el procesador quede completamente libre durante las transferencias, relegando toda la lógica de serialización y control de protocolo a la FPGA. Esta migración es factible sin comprometer los recursos disponibles: con 14 transceptores instanciados simultáneamente, la utilización de la FPGA se sitúa en torno al 7% de los recursos totales disponibles, dejando margen más que suficiente para absorber la lógica adicional del controlador DMA y los FIFOs AXI-Stream en la PL.

Integración con el stack de software de vuelo de Sener. El driver y las PCBs han sido diseñados con los requisitos de Indra/Sener como guía. El siguiente paso natural es la integración del driver en el stack de software de vuelo de Sener y la realización de pruebas de validación conjuntas en entorno *Hardware-in-the-Loop*.

Soporte de protocolos adicionales. La arquitectura modular del transceptor VHDL facilita la incorporación de nuevos modos de operación. Una extensión interesante sería añadir soporte nativo del protocolo MIL-STD-1553, ampliamente utilizado en sistemas espaciales de mayor herencia.

Bibliografía

- [1] Indra. Proyecto LINCE: Línea de industrialización de cargas de pago y plataformas espaciales. Indra Sistemas S.A., 2026. Recuperado el 12 de junio de 2026.
- [2] VITA Standards Organization. VITA 57.1: FPGA Mezzanine Card (FMC) standard. VITA Standards Organization, 2010.
- [3] Texas Instruments. The RS-485 design guide. Application Report SLLA272C, Texas Instruments, 2014.
- [4] Electronic Industries Alliance (EIA). Electrical characteristics of generators and receivers for use in balanced digital multipoint systems. TIA/EIA-485-A, 1998.
- [5] Texas Instruments. RS-422 and RS-485 standards overview and system configurations. Application Report SLLA070D, Texas Instruments, 2018.
- [6] Telecommunications Industry Association. Electrical characteristics of balanced voltage digital interface circuits. TIA/EIA-422-B, 1994.
- [7] European Cooperation for Space Standardization (ECSS). Space engineering – SpaceWire – links, nodes, routers and networks, 2008.
- [8] S. M. Parkes. *SpaceWire User's Guide*. STAR-Dundee Ltd., 2012.
- [9] International Organization for Standardization (ISO). Road vehicles — controller area network (CAN) — part 1: Data link layer and physical signalling. ISO 11898-1:2015, 2015.
- [10] Agencia Espacial Europea (ESA). CAN in space: Implementation guide and application notes. Technical report, TEC-ED & TEC-SW, ESA, 2015.
- [11] Texas Instruments. ADS7950/51/52/53 — 12/10/8-bit, 1-MSPS, 4-/8-channel, serial interface, MicroPower sampling analog-to-digital converter. Technical Report SLAS605C, Texas Instruments, 2018.
- [12] AMD/Xilinx. *Zynq UltraScale+ MPSoC Technical Reference Manual*. AMD/Xilinx, 2023.

Anexo 1: Mapas de señales y tabla NCO

Este anexo recoge el mapa de señales completo entre los conectores FMC de la ZCU102 y las placas LINCE Comunicación Serial, CDHS y AOCS, así como la tabla de frecuencias soportadas por el NCO del transceptor serie junto con su error medido.

A1.1 Mapa de señales LINCE Comunicación Serial

La siguiente tabla recoge el mapeado de los 14 drivers RS-485/RS-422 de la placa LINCE Comunicación Serial con el conector FMC HPC0 (P2) de la ZCU102. Cada driver expone las señales TX, RX, SLO y DE (esta última ausente en los drivers 7 y 11, que sólo disponen de control de dirección hardware).

Driver	Señal	Red Interna	Red (FMC HPC0)	Pin FMC (P2)	Pin Zynq
Driver0	TX	TX0	FMC_HPC0_LA32_N	H38	T11
Driver0	RX	RX0	FMC_HPC0_LA26_N	D27	K15
Driver0	SLO	SLO0	FMC_HPC0_LA27_N	C27	L10
Driver0	DE	DE0	FMC_HPC0_LA33_N	G37	V11
Driver1	TX	TX1	FMC_HPC0_LA33_P	G36	V12
Driver1	RX	RX1	FMC_HPC0_LA30_N	H35	U6
Driver1	SLO	SLO1	FMC_HPC0_LA32_P	H37	U11
Driver1	DE	DE1	FMC_HPC0_LA27_P	C26	M10
Driver2	TX	TX2	FMC_HPC0_LA30_P	H34	V6
Driver2	RX	RX2	FMC_HPC0_LA26_P	D26	L15
Driver2	SLO	SLO2	FMC_HPC0_LA31_N	G34	V7
Driver2	DE	DE2	FMC_HPC0_LA31_P	G33	V8
Driver3	TX	TX3	FMC_HPC0_LA29_N	G31	U8
Driver3	RX	RX3	FMC_HPC0_LA29_P	G30	U9
Driver3	SLO	SLO3	FMC_HPC0_LA28_N	H32	T6
Driver3	DE	DE3	FMC_HPC0_LA28_P	H31	T7
Driver4	TX	TX4	FMC_HPC0_LA14_P	C18	AC7
Driver4	RX	RX4	FMC_HPC0_LA13_N	D18	AC8
Driver4	SLO	SLO4	FMC_HPC0_LA22_P	G24	M15

Driver	Señal	Red Interna	Red (FMC HPC0)	Pin FMC (P2)	Pin Zynq
Driver4	DE	DE4	FMC_HPC0.LA19_N	H23	K13
Driver5	TX	TX5	FMC_HPC0.LA20_N	G22	M13
Driver5	RX	RX5	FMC_HPC0.LA15_N	H20	Y9
Driver5	SLO	SLO5	FMC_HPC0.LA19_P	H22	L13
Driver5	DE	DE5	FMC_HPC0.LA20_P	G21	N13
Driver6	TX	TX6	FMC_HPC0.LA15_P	H19	Y10
Driver6	RX	RX6	FMC_HPC0.LA13_P	D17	AB8
Driver6	SLO	SLO6	FMC_HPC0.LA16_N	G19	AA12
Driver6	DE	DE6	FMC_HPC0.LA16_P	G18	Y12
Driver7	TX	TX7	FMC_HPC0.LA06_P	C10	AC2
Driver7	RX	RX7	FMC_HPC0.LA01_CC_N	D9	AC4
Driver7	SLO	SLO7	FMC_HPC0.LA01_CC_P	D8	AB4
Driver8	TX	TX8	FMC_HPC0.LA05_P	D11	AB3
Driver8	RX	RX8	FMC_HPC0.LA05_N	D12	AC3
Driver8	SLO	SLO8	FMC_HPC0.LA06_N	C11	AC1
Driver8	DE	DE8	FMC_HPC0.LA10_P	C14	W5
Driver9	TX	TX9	FMC_HPC0.LA02_P	H7	V2
Driver9	RX	RX9	FMC_HPC0.LA00_CC_N	G7	Y3
Driver9	SLO	SLO9	FMC_HPC0.LA00_CC_P	G6	Y4
Driver9	DE	DE9	FMC_HPC0.LA02_N	H8	V1
Driver10	TX	TX10	FMC_HPC0.LA09_P	D14	W2
Driver10	RX	RX10	FMC_HPC0.LA03_N	G10	Y1
Driver10	SLO	SLO10	FMC_HPC0.LA03_P	G9	Y2
Driver10	DE	DE10	FMC_HPC0.LA04_P	H10	AA2
Driver11	TX	TX11	FMC_HPC0.LA08_N	G13	V3
Driver11	RX	RX11	FMC_HPC0.LA08_P	G12	V4
Driver11	SLO	SLO11	FMC_HPC0.LA04_N	H11	AA1
Driver12	TX	TX12	FMC_HPC0.LA10_N	C15	W4
Driver12	RX	RX12	FMC_HPC0.LA07_N	H14	U4
Driver12	SLO	SLO12	FMC_HPC0.LA07_P	H13	U5
Driver12	DE	DE12	FMC_HPC0.LA09_N	D15	W1
Driver13	TX	TX13	FMC_HPC0.LA11_N	H17	AB5
Driver13	RX	RX13	FMC_HPC0.LA11_P	H16	AB6
Driver13	SLO	SLO13	FMC_HPC0.LA12_P	G15	W7
Driver13	DE	DE13	FMC_HPC0.LA12_N	G16	W6

Tabla 5.1: Mapa de señales entre la placa LINCE Comunicación Serial y el conector FMC HPC0 (P2) de la ZCU102.

A1.2 Mapa de señales CDHS

Subsistema	Señal	Red (FMC HPC0)	Pin FMC	Pin Zynq
SPI (ADC)	SDO	FMC_HPC0_LA02_N	H8	V1
SPI (ADC)	SDI	FMC_HPC0_LA00_CC_N	G7	Y3
SPI (ADC)	SCLK	FMC_HPC0_LA02_P	H7	V2
SPI (ADC)	CS_N	FMC_HPC0_LA00_CC_P	G6	Y4
CAN (Nominal)	TX_N	FMC_HPC0_LA21_P	H25	P12
CAN (Nominal)	RX_N	FMC_HPC0_LA22_P	G24	M15
CAN (Redundante)	TX_R	FMC_HPC0_LA21_N	H26	N12
CAN (Redundante)	RX_R	FMC_HPC0_LA22_N	G25	M14
RS1 (Serial 1)	TX	FMC_HPC0_LA15_P	H19	Y10
RS1 (Serial 1)	RX	FMC_HPC0_LA11_N	H17	AB5
RS1 (Serial 1)	DE	FMC_HPC0_LA16_P	G18	Y12
RS2 (Serial 2)	TX	FMC_HPC0_LA12_N	G16	W6
RS2 (Serial 2)	RX	FMC_HPC0_LA12_P	G15	W7
RS2 (Serial 2)	DE	FMC_HPC0_LA11_P	H16	AB6
RS3 (Serial 3)	TX	FMC_HPC0_LA07_N	H14	U4
RS3 (Serial 3)	RX	FMC_HPC0_LA07_P	H13	U5
RS3 (Serial 3)	DE	FMC_HPC0_LA08_N	G13	V3
PWM (Heaters)	IN1	FMC_HPC0_LA04_N	H11	AA1
PWM (Heaters)	IN2	FMC_HPC0_LA04_P	H10	AA2
PWM (Heaters)	IN3	FMC_HPC0_LA03_P	G9	Y2
PWM (Heaters)	IN4	FMC_HPC0_LA03_N	G10	Y1

Tabla 5.2: Mapa de señales entre la placa CDHS y el conector FMC HPC0 de la ZCU102.

A1.3 Mapa de señales AOCs

Subsistema	Señal	Red (FMC HPC0)	Pin FMC	Pin Zynq
RS1 (Serial J1)	TX	FMC_HPC0_LA20_N	G22	M13
RS1 (Serial J1)	RX	FMC_HPC0_LA20_P	G21	N13
RS1 (Serial J1)	DE	FMC_HPC0_LA19_P	H22	L13
MOT-PWM (J2)	PWM_X_1	FMC_HPC0_LA17_CC_N	D21	N11
MOT-PWM (J2)	PWM_X_2	FMC_HPC0_LA15_N	H20	Y9
MOT-PWM (J2)	PWM_Y_1	FMC_HPC0_LA16_N	G19	AA12
MOT-PWM (J2)	PWM_Y_2	FMC_HPC0_LA15_P	H19	Y10
MOT-PWM (J2)	PWM_Z_1	FMC_HPC0_LA16_P	G18	Y12
MOT-PWM (J2)	PWM_Z_2	FMC_HPC0_LA13_N	D18	AC8
RS3 (Serial J3)	TX	FMC_HPC0_LA11_N	H17	AB5
RS3 (Serial J3)	RX	FMC_HPC0_LA12_P	G15	W7
RS3 (Serial J3)	DE	FMC_HPC0_LA11_P	H16	AB6
RS4 (Serial J4)	TX	FMC_HPC0_LA09_P	D14	W2
RS4 (Serial J4)	RX	FMC_HPC0_LA08_N	G13	V3
RS4 (Serial J4)	DE	FMC_HPC0_LA07_N	H14	U4
RS5 (Serial J5)	TX	FMC_HPC0_LA07_P	H13	U5
RS5 (Serial J5)	RX	FMC_HPC0_LA05_P	D11	AB3
RS5 (Serial J5)	DE	FMC_HPC0_LA08_P	G12	V4
SPW1 (J6)	DIN_P	FMC_HPC0_LA02_P	H7	V2
SPW1 (J6)	DIN_N	FMC_HPC0_LA02_N	H8	V1
SPW1 (J6)	SIN_P	FMC_HPC0_LA04_P	H10	AA2
SPW1 (J6)	SIN_N	FMC_HPC0_LA04_N	H11	AA1
SPW1 (J6)	SOUT_P	FMC_HPC0_LA01_CC_P	D8	AB4
SPW1 (J6)	SOUT_N	FMC_HPC0_LA01_CC_N	D9	AC4
SPW1 (J6)	DOUT_P	FMC_HPC0_LA00_CC_P	G6	Y4
SPW1 (J6)	DOUT_N	FMC_HPC0_LA00_CC_N	G7	Y3
SPW2 (J7)	DIN_P	FMC_HPC0_LA21_P	H25	P12
SPW2 (J7)	DIN_N	FMC_HPC0_LA21_N	H26	N12
SPW2 (J7)	SIN_P	FMC_HPC0_LA24_P	H28	L12
SPW2 (J7)	SIN_N	FMC_HPC0_LA24_N	H29	K12
SPW2 (J7)	SOUT_P	FMC_HPC0_LA28_P	H31	T7
SPW2 (J7)	SOUT_N	FMC_HPC0_LA28_N	H32	T6
SPW2 (J7)	DOUT_P	FMC_HPC0_LA30_P	H34	V6
SPW2 (J7)	DOUT_N	FMC_HPC0_LA30_N	H35	U6
RS8 (Serial J8)	TX	FMC_HPC0_LA22_N	G25	M14
RS8 (Serial J8)	RX	FMC_HPC0_LA19_N	H23	K13
RS8 (Serial J8)	DE	FMC_HPC0_LA22_P	G24	M15

Tabla 5.3: Mapa de señales entre la placa AOCs y el conector FMC HPC0 de la ZCU102.

A1.4 Tabla de frecuencias del NCO

La siguiente tabla recoge las 54 frecuencias de baudrate soportadas por el oscilador controlado numéricamente (NCO) de 32 bits del transceptor serie, junto con los valores de incremento configurados y el error medido en ppm respecto a la frecuencia nominal.

Baud	Half	INC_ROM	INC_HALF_ROM	Medido (Hz)	Error (ppm)
9600	0	412317	824634	9600,002	0,2
14400	0	618475	1236951	14399,988	-0,8
19200	0	824634	1649267	19200,012	0,6
28800	0	1236951	2473901	28799,977	-0,8
31250	0	1342177	2684355	31250,000	0,0
38400	0	1649267	3298535	38400,025	0,6
56000	0	2405182	4810363	55999,978	-0,4
57600	0	2473901	4947802	57600,037	0,6
74400	0	3195456	6390911	74400,057	0,8
115200	0	4947802	9895605	115200,074	0,6
128000	0	5497558	10995116	128000,000	0,0
153600	0	6597070	13194140	153600,393	2,6
230400	0	9895605	19791209	230398,820	-5,1
256000	0	10995116	21990233	256000,000	0,0
312500	0	13421773	26843546	312500,000	0,0
460800	0	19791209	39582419	460808,258	17,9
500000	0	21474836	42949673	500000,000	0,0
576000	0	24739012	49478023	576003,686	6,4
614400	0	26388279	52776558	614401,573	2,6
750000	0	32212255	64424509	749990,625	-12,5
921600	0	39582419	79164837	921616,515	17,9
1000000	0	42949673	85899346	1000000,000	0,0
1152000	0	49478023	98956046	1152007,373	6,4
1500000	0	64424509	128849019	1499925,004	-50,0
1843200	0	79164837	158329674	1843148,097	-28,2
2000000	0	85899346	171798692	2000000,000	0,0
2500000	0	107374182	214748365	2500000,000	0,0
3000000	0	128849019	257698038	3000300,030	100,0
3686400	0	158329674	316659349	3685956,506	-120,3

Tabla 5.4: Tabla NCO: frecuencias de baudrate soportadas y error medido (extracto, modo *full-bit*).

Anexo A: Aspectos éticos, económicos, sociales y ambientales

El apartado “Requisitos de las acreditaciones internacionales EUR-ACE y ABET” de la Normativa de TFT de la ETSIT-UPM establece que *“La memoria del TFT del GITST, GIB y MUIT, y en general la de aquellas titulaciones que hayan obtenido o para las que se desee solicitar una acreditación internacional EUR-ACE o ABET, debe mostrar conciencia de la responsabilidad de la aplicación práctica de la ingeniería, el impacto social y ambiental, el compromiso con la ética profesional, la responsabilidad y las normas de la aplicación práctica de la ingeniería, así como sobre las prácticas de gestión de proyectos, gestión, y control de riesgos, entendiendo sus limitaciones”*.

Este anexo obligatorio del TFT tendrá un carácter sintético con los siguientes apartados:

A1. INTRODUCCIÓN

Este trabajo se enmarca en el proyecto LINCE, una iniciativa del PERTE Aeroespacial para el desarrollo de microsátélites LEO liderada por Indra. El objetivo es desarrollar el subsistema de comunicaciones serie del ordenador de a bordo (OBC) del satélite, cubriendo tanto el diseño de hardware electrónico (PCBs de prueba) como el firmware embebido sobre un sistema operativo de tiempo real. Este tipo de tecnología tiene implicaciones relevantes en ámbitos sociales, económicos, ambientales y éticos que se analizan a continuación.

A2. DESCRIPCIÓN DE IMPACTOS RELEVANTES RELACIONADOS CON EL PROYECTO

[Pendiente de redacción — describir impactos en: soberanía tecnológica espacial, doble uso civil/militar de la tecnología satelital, residuos electrónicos (PCBs), consumo energético de la FPGA, implicaciones de los sistemas de observación LEO.]

A3. ANÁLISIS DETALLADO DE ALGUNO DE LOS IMPACTOS

[Pendiente de redacción — análisis en profundidad del impacto seleccionado.]

A4. CONCLUSIONES

[Pendiente de redacción — valoración del proyecto desde el punto de vista ético, social, económico y medioambiental.]

Anexo B: Presupuesto económico

El apartado “Requisitos de las acreditaciones internacionales EUR-ACE y ABET” de la Normativa de TFT de la ETSIT-UPM establece que “*La memoria del TFT del GITST, GIB y MUIT, y en general la de aquellas titulaciones que hayan obtenido o para las que se desee solicitar una acreditación internacional EUR-ACE o ABET, ... debe incluir un presupuesto económico*”. A modo de ejemplo, la tabla del presupuesto de un proyecto podría ser la siguiente:

COSTE DE MANO DE OBRA (coste directo)		Horas	Precio/hora	Total
		300	15 €	4.500 €
COSTE DE RECURSOS MATERIALES (coste directo)	Precio de compra	Uso en meses	Amortización (en años)	Total
Ordenador personal (Software incluido).....	1.500,00 €	6	5	150,00 €
Impresora láser	500,00 €	6	5	50,00 €
Otro equipamiento				
COSTE TOTAL DE RECURSOS MATERIALES				200,00 €
GASTOS GENERALES (costes indirectos)	15 %	sobre CD		705,00 €
BENEFICIO INDUSTRIAL	6 %	sobre CD + CI		324,30 €
MATERIAL FUNGIBLE				
Impresión				100,00 €
Encuadernación				300,00 €
SUBTOTAL PRESUPUESTO				6.129,30 €
IVA APLICABLE			21 %	1.287,15 €
TOTAL PRESUPUESTO				7.416,45 €

Tabla 5.5: Presupuesto económico